

[TO BE COMPLETED: University or engineering school]

[TO BE COMPLETED: Degree]

Specialty: Software Engineering and Information Systems

University logo
placeholder

Host organization
logo placeholder

PerTrack: Performance Management and HR Evaluation Platform

Graduation Engineering Report

Student: **[TO BE COMPLETED: Student name]**

Host organization: **[TO BE COMPLETED: Host organization]**

Academic supervisor: **[TO BE COMPLETED: Academic supervisor name]**

Professional supervisor: **[TO BE COMPLETED: Professional supervisor name]**

Academic year: 2025–2026

Repository: `application_gestion_competences`

Report generated from repository evidence on July 21, 2026

Report Validation Page

This report documents the software project identified in the repository as a performance management and HR evaluation platform. The technical content was prepared from the application source code, configuration files, tests, Docker and Kubernetes manifests, AI service artifacts, and existing repository documentation.

Student name	[TO BE COMPLETED: Student name]
Academic supervisor	[TO BE COMPLETED: Academic supervisor name]
Professional supervisor	[TO BE COMPLETED: Professional supervisor name]
Host organization	[TO BE COMPLETED: Host organization]
Validation date	[TO BE COMPLETED: Validation date]
Signature	[TO BE COMPLETED: Signature]

Dedication

[TO BE COMPLETED: Personal dedication, if required by the institution.]

Acknowledgements

I would like to thank my academic supervisor, my professional supervisor, and everyone who contributed guidance, feedback, and review during this project. I also thank the future users represented by the business requirements of the platform: administrators, HR officers, team leaders, and collaborators. Their workflows shaped the main design choices of this performance management system.

Several personal and institutional details were not available in the repository and are therefore kept as placeholders rather than invented.

Abstract

PerTrack is a web-based performance management and HR evaluation platform. The implemented system centralizes annual performance cycles, objective planning, KPI tracking, team and subteam organization, task follow-up, mid-year check-ins, final evaluations, HR validation, improvement plans, bonus and penalty decisions, career recommendations, analytics, audit logs, notifications, and AI-assisted review support. The repository shows a React and Vite single-page frontend, a Node.js and Express API backed by MongoDB through Mongoose, a Python Flask AI microservice using Random Forest and XGBoost artifacts, and deployment assets for Docker, Kubernetes, Kustomize, Argo CD, and GitHub Actions.

The report analyzes the project from business, functional, architectural, implementation, security, testing, AI, and DevOps perspectives. It uses repository evidence rather than assumed functionality. The analysis highlights the main engineering challenge of the project: transforming a sensitive HR process into an auditable workflow with phase controls, role-based access, objective weight constraints, manager review, HR validation, and explicit limitations around AI-generated content and synthetic model training data.

Keywords: performance management, HR evaluation, MERN stack, MongoDB, React, Express, AI-assisted review, XGBoost, Kubernetes, GitOps.

Resume

PerTrack est une application web de gestion de la performance et d'évaluation RH. Le systeme implemente permet de centraliser les cycles annuels, les objectifs, les KPI, les equipes, les taches, les points de suivi, les evaluations finales, la validation RH, les plans d'amelioration, les decisions de bonus ou de penalite, les recommandations de carriere, les tableaux de bord, les journaux d'audit, les notifications et certaines fonctions d'assistance par intelligence artificielle.

Le projet repose sur une application frontend React avec Vite, une API backend Node.js et Express, une base MongoDB modelisee avec Mongoose, et un service Python Flask pour les predictions de performance. Le depot contient aussi des artefacts Docker, Kubernetes, Kustomize, Argo CD et GitHub Actions. Ce rapport presente l'analyse fonctionnelle, technique, architecturale, securite, test, IA et DevOps du projet en s'appuyant uniquement sur les elements verifies dans le depot.

Mots-cles: gestion de la performance, evaluation RH, MERN, MongoDB, React, Express, intelligence artificielle, XGBoost, Kubernetes, GitOps.

List of Abbreviations and Acronyms

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Delivery
HR	Human Resources
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MERN	MongoDB, Express, React, Node.js
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
XSS	Cross-Site Scripting

Contents

Report Validation Page	i
Dedication	ii
Acknowledgements	iii
Abstract	iv
Resume	v
List of Abbreviations and Acronyms	vi
General Introduction	1
1 General Context and Project Environment	4
1.1 Introduction	4
1.2 Business Domain	4
1.3 Stakeholders and Target Users	5
1.4 Project Identity and Repository Evidence	5
1.5 Development Approach	6
1.6 Version Control and Quality Culture	7
1.7 Chapter Conclusion	7
2 Project Analysis, Existing Solutions, and Planning	8
2.1 Problem Statement	8
2.2 Project Objectives	8
2.3 Scope	9
2.4 Comparable Solutions	9
2.5 Risk Analysis	10
2.6 Planning by Sprints	12
2.7 Chapter Conclusion	12

3	Requirements and System Architecture	13
3.1	Functional Requirements	13
3.2	Non-Functional Requirements	14
3.3	Architecture	15
3.4	Technology Choices	16
3.5	API Architecture	18
3.6	Chapter Conclusion	18
4	Sprint 1: Organization, Access Control, and Cycle Foundation	19
4.1	Sprint Objective	19
4.2	Authentication	19
4.3	Role-Based Access Control	20
4.4	Team and Subteam Management	21
4.5	Cycle Management	22
4.6	Security Controls Introduced in This Sprint	22
4.7	Sprint Result	22
4.8	Design Decisions and Trade-offs	22
5	Sprint 2: Objectives, KPIs, Tasks, and Check-Ins	24
5.1	Sprint Objective	24
5.2	Objective Model	24
5.3	Objective Lifecycle	24
5.4	Weight Rules	26
5.5	Team Objective Weighting	27
5.6	KPI Management	27
5.7	Tasks	27
5.8	Check-Ins	27
5.9	Sprint Result	28
5.10	Implementation Challenges	28
6	Sprint 3: Evaluation, HR Review, and Follow-Up Decisions	29
6.1	Sprint Objective	29
6.2	Two Evaluation Layers	29
6.3	Automatic Score and Manager Adjustment	30
6.4	HR Validation	30
6.5	Improvement Plans	31
6.6	HR Decisions and Bonus/Penalty Records	31
6.7	Career Development	32
6.8	PDF and Report Exports	32
6.9	Sprint Result	32

6.10	Design Decisions and Trade-offs	32
7	Sprint 4: AI Assistance, Prediction, Analytics, and Auditability	34
7.1	Sprint Objective	34
7.2	Generative AI Service	34
7.3	AI Controller Capabilities	34
7.4	Python Prediction Service	35
7.5	Prediction Formula	36
7.6	Dataset and Model Evidence	36
7.7	Analytics	38
7.8	Auditability	38
7.9	AI Limitations and Ethics	38
7.10	Sprint Result	38
7.11	Design Decisions and Trade-offs	39
8	Sprint 5: Testing, Continuous Integration, and Deployment	40
8.1	Sprint Objective	40
8.2	Testing Strategy	40
8.3	Backend Test Coverage by Behavior	41
8.4	Frontend Test Coverage by Behavior	41
8.5	Dockerization	41
8.6	Kubernetes and GitOps	42
8.7	CI/CD Pipeline	42
8.8	Deployment Limitations	43
8.9	Chapter Conclusion	44
	General Conclusion and Perspectives	45
A	API Summary	47
B	Database Dictionary	49
C	Frontend Module Inventory	51
C.1	Route-Level Pages	51
C.2	Shared Application Shell	52
C.3	Authentication and API Clients	53
C.4	Goal and Objective UI	53
C.5	Work Management UI	53
C.6	Evaluation UI	54
C.7	UI States	54

D Business Rule Catalog	55
D.1 Role and Identity Rules	55
D.2 Team and Scope Rules	55
D.3 Cycle Rules	56
D.4 Objective Workflow Rules	57
D.5 Scoring Rules	57
D.6 HR Review Rules	58
E Security Review	60
E.1 Implemented Controls	60
E.2 Authentication Risks	61
E.3 Authorization Risks	61
E.4 Data Privacy Risks	61
E.5 AI Privacy Risks	62
E.6 Deployment Risks	62
E.7 Residual Risk Summary	62
F AI Model Evidence	63
F.1 Dataset Fields	63
F.2 Dataset Quality Checks	64
F.3 Class Distribution	64
F.4 Promotion Readiness by Rating	64
F.5 Rating Confusion Matrix	65
F.6 Promotion Confusion Matrix	65
F.7 Feature Importance	65
F.8 Interpretation	65
G Installation Guide	67
G.1 Prerequisites	67
G.2 Backend	67
G.3 Frontend	67
G.4 AI Service	67
G.5 Docker Compose	68
H Test Evidence	69
H.1 Backend Tests	69
H.2 Frontend Tests	69
H.3 Frontend Build	69
H.4 AI Artifact Evaluation	69

List of Figures

1	Performance-management lifecycle implemented by the application. Source: author's own work based on cycle phases and business modules. . .	1
1.1	System-context diagram. Source: author's own work based on repository architecture.	6
1.2	Repository workflow from local implementation to CI/CD evidence. Source: author's own work based on repository folders and GitHub Actions. .	6
2.1	Project scope boundary and verified limitations. Source: author's own work based on repository evidence.	9
2.2	Indicative project planning reconstructed from repository history and module maturity. Source: author's own work; exact internship dates are not present in the repository.	12
3.1	Global use-case diagram. Source: author's own work based on routes and frontend route configuration.	15
3.2	Logical architecture. Source: author's own work based on application source code.	15
3.3	Backend component diagram. Source: author's own work based on Express route and controller structure.	16
3.4	Frontend, backend, database, and AI communication flow. Source: author's own work based on API client, routes, services, and AI files.	16
3.5	Database model overview. Source: author's own work based on Mongoose schemas.	16
3.6	Verified technology stack grouped by project responsibility. Source: author's own work based on manifests and deployment files.	17
4.1	Authentication and token-refresh sequence. Source: author's own work based on 'AuthContext.jsx', 'api.js', and backend auth middleware.	19
4.2	Frontend route-guard decision flow. Source: author's own work based on 'RouteGuard.jsx' and route metadata.	21

4.3	Performance-cycle state diagram. Source: author's own work based on <code>backend/routes/cycles.js</code>	22
5.1	Objective-submission sequence diagram. Source: author's own work based on objective routes and controller workflow.	25
5.2	Global workflow activity diagram. Source: author's own work based on implemented modules.	25
5.3	Objective scoring and weight-normalization logic. Source: author's own work based on scoring and objective-rule utilities.	26
5.4	KPI and scoring flow. Source: author's own work based on objective, check-in, and scoring modules.	27
5.5	Check-in review workflow. Source: author's own work based on 'CheckIn.js' and check-in routes.	28
6.1	Final evaluation sequence diagram. Source: author's own work based on final evaluation controller and scoring service.	30
6.2	HR-review sequence diagram. Source: author's own work based on HR validation, decision, and improvement-plan modules.	31
7.1	AI architecture separating generative drafting from predictive scoring. Source: author's own work based on backend AI services and the Flask AI service.	35
7.2	AI inference sequence diagram. Source: author's own work based on Node AI service and Flask API.	35
7.3	Model comparison from saved artifact evaluation. Source: project AI artifacts evaluated locally.	37
7.4	Illustrative feature-importance ranking from the saved XGBoost rating model evaluation. Source: project AI artifacts evaluated locally.	37
8.1	Testing strategy map distinguishing verified tests from missing evidence. Source: repository tests and local verification.	41
8.2	Deployment architecture. Source: author's own work based on Docker, Nginx, and Kubernetes manifests.	42
8.3	CI/CD pipeline diagram. Source: author's own work based on GitHub Actions workflow.	42

List of Tables

1.1	Stakeholders identified from implemented roles and workflows.	5
2.1	Comparative analysis by feature category.	10
2.2	Project risk matrix.	11
3.1	Functional requirements verified from source code.	13
3.2	Non-functional requirements and implementation evidence.	14
3.3	Technology-choice table based on package manifests.	17
4.1	Role-permission matrix. Source: author’s own work based on backend role middleware and route configuration.	20
6.1	Final score rating bands implemented in <code>scoreCalculationService.js</code> . .	30
6.2	Examples of HR review controls implemented in workflow rules.	31
7.1	Model evaluation on the saved artifacts using the repository training split.	36
8.1	Local verification results from July 21, 2026.	40
8.2	CI/CD stages verified from the GitHub Actions workflow.	43
8.3	Delivered scope and remaining limitations.	46
A.1	Grouped API inventory.	47
B.1	Main MongoDB/Mongoose entities.	49
C.1	Frontend route inventory.	51
C.2	Frontend service and utility modules.	53
D.1	Role and identity rules.	55
D.2	Team and access-scope rules.	56
D.3	Cycle and phase rules.	56
D.4	Objective workflow and visibility rules.	57
D.5	Scoring rules.	57
D.6	HR review and follow-up rules.	58

E.1	Implemented security controls.	60
E.2	Residual security risks and recommendations.	62
F.1	AI dataset and prediction fields.	63
F.2	Dataset checks computed locally.	64
F.3	Rating class distribution in the synthetic dataset.	64
F.4	Promotion-readiness rate by rating in the synthetic data.	64
F.5	XGBoost rating model confusion matrix on the repository split.	65
F.6	XGBoost promotion model confusion matrix on the repository split.	65
F.7	XGBoost rating feature importance from saved artifact.	65

General Introduction

Performance evaluation becomes difficult when goals, progress evidence, comments, and HR decisions are scattered across documents, spreadsheets, private messages, and late review meetings. A manager may know that an employee worked hard, but the organization needs a traceable answer to more precise questions: what objectives were agreed, which KPIs were used, who approved them, how much progress was recorded, whether the employee had a chance to assess their own work, and how HR validated the final decision. The PerTrack project addresses that operational problem by turning a sensitive annual review process into a structured web application.

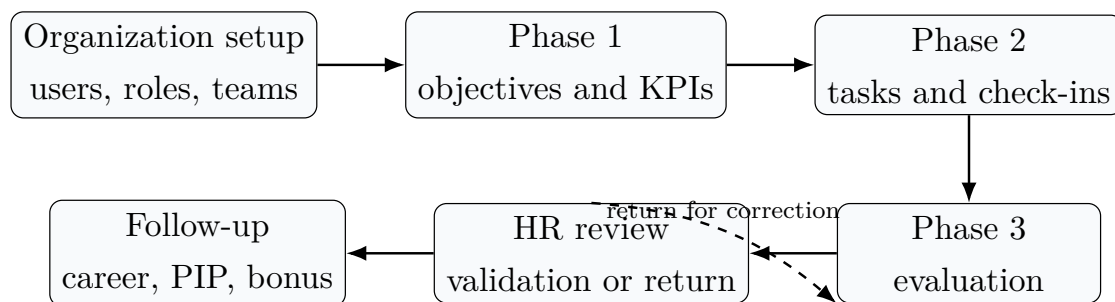


Figure 1: Performance-management lifecycle implemented by the application. Source: author’s own work based on cycle phases and business modules.

The lifecycle in [figure 1](#) is the backbone of the report. It shows why the application cannot be reduced to isolated CRUD screens: each module prepares evidence for the next step. A cycle creates time boundaries, objectives define the performance contract, tasks and check-ins capture execution, evaluations transform evidence into judgment, and HR review gives the process institutional control.

The repository shows a system built around four roles: **ADMIN**, **HR**, **TEAM_LEADER**, and **COLLABORATOR**. These roles are not decorative labels. They control route access in the backend, menu access in the frontend, and the visibility of objectives, evaluations, HR decisions, and audit information. This is important because performance data is personal and organizationally sensitive. A collaborator should be able to draft objectives and follow their own evaluation, while a team leader should review managed employees, and HR should validate the final process without bypassing every manager-owned step.

The application also gives special attention to objective weights. In this project,

weights are not merely display values. They define how objective achievement contributes to the automatic evaluation score. A team objective is not divided by the number of members; instead, the full weight consumes capacity for every assigned member. This design is visible in both the scoring service and its tests. It prevents a shared objective from becoming artificially small when many employees participate, while still requiring the system to control the total allocation per employee and per team bucket.

PerTrack combines a React and Vite frontend with an Express and MongoDB backend. The frontend is organized around protected routes, role-based menus, dashboards, modal workflows, charts, kanban task views, active-cycle awareness, and token refresh logic. The backend exposes REST endpoints for users, teams, cycles, objectives, check-ins, tasks, meetings, feedback, evaluations, final evaluations, HR decisions, improvement plans, career recommendations, reports, notifications, calendar integration, audit logs, and AI assistance. MongoDB is used through Mongoose schemas with references and embedded arrays, which fits the project because objectives contain KPIs, comments, activity logs, change requests, and attachments that evolve during the review lifecycle [1], [2].

The project includes two forms of AI support. The first is generative assistance in the Node.js backend: goal suggestions, KPI suggestions, objective refinement, check-in drafts, mid-year review drafts, final self-assessment drafts, manager review drafts, and development-plan suggestions. The implementation is provider-aware and can use Gemini, xAI/Grok, or OpenAI-compatible chat completions depending on environment variables. The second is a Python Flask microservice trained on a synthetic dataset for performance rating and promotion-readiness prediction. That AI service loads saved XGBoost model artifacts, validates numeric inputs, computes a deterministic overall score, returns a predicted rating and promotion probability, and generates textual strengths, weaknesses, summaries, and suggestions. Because the dataset is synthetic, the report treats model metrics as project evidence, not as proof of real-world HR validity.

The engineering objective of the project is therefore wider than a simple CRUD application. The system must coordinate roles, workflow phases, objective approval, team hierarchy, self-assessment, manager scoring, HR review, auditability, and deployment. The implemented DevOps assets confirm this ambition: Dockerfiles for backend and frontend, a Compose file for local containers, Kubernetes base manifests, Kustomize overlays for dev, QA, staging, and production, an Argo CD application, and a GitHub Actions workflow with secret scanning, backend tests, frontend build/tests, Docker image build and push, Trivy scans, GitOps tag update, and a health-check smoke test.

This report follows the project from context to implementation. Chapter 1 presents the project environment and stakeholders. Chapter 2 studies the business problem, project scope, risks, and planning. Chapter 3 defines requirements and architecture. Chapters 4 to 7 follow implementation sprints covering organization/access, objectives/KPIs/tasks, evaluation/HR workflows, and AI/analytics. Chapter 8 covers testing, DevOps, deploy-

ment, and quality evidence. The report ends with limitations and realistic future work.

Chapter 1

General Context and Project Environment

1.1 Introduction

The repository does not contain the name of the university, the student, the host organization, or the supervisors. Those fields are therefore kept as placeholders. What can be verified is the nature of the software project: an HR-oriented performance management platform developed as a final-year engineering project and stored in a repository named `application_gestion_competences`. Several files also use names such as `pfe`, `PerTrack`, and `HR Evaluation System`. The most precise product name used in this report is `PerTrack`, because the repository contains `PerTrack` diagram files and the AI service README describes integration with the `PerTrack` MERN app.

1.2 Business Domain

The business domain is employee performance management. In practice, this domain joins administrative processes and management practice: an organization defines a cycle, employees and managers agree on objectives, progress is monitored through tasks and check-ins, managers evaluate results, and HR validates the process before decisions such as improvement plans, bonuses, penalties, promotion recommendations, or career actions are recorded.

The project implements this domain with explicit data entities rather than informal notes. `Cycle` stores the annual review period and three phases. `Objective` stores goals, weights, KPIs, approval status, self-assessment, manager adjustment, comments, change requests, and activity logs. `FinalEvaluation` stores the automatic score, manager score, final score, rating label, objective breakdown, evidence summary, HR status, workflow history, and employee feedback. This model shows that the project is built around gov-

ernance and traceability, not only dashboards.

1.3 Stakeholders and Target Users

Table 1.1: Stakeholders identified from implemented roles and workflows.

Stakeholder	Verified role or evidence	Main expectation
Administrator	<code>ADMIN</code> role in <code>User.js</code> and route guards	Configure users, cycles, teams, global access, analytics, audit logs, and deployment-level administration.
HR officer	<code>HR</code> role in routes and final evaluation workflow	Review manager-submitted evaluations, validate or return them, create HR decisions, follow up development actions, and consult analytics.
Team leader	<code>TEAM_LEADER</code> role and access-control helpers	Manage team/subteam scope, approve objectives, review check-ins, generate evaluations, and supervise tasks.
Collaborator	<code>COLLABORATOR</code> role and collaborator routes	Draft objectives, submit check-ins, perform self-assessment, view feedback, and acknowledge evaluation outputs.
Academic jury	Report requirement and project documentation	Understand the engineering choices, implemented scope, limitations, tests, and deployment approach.

1.4 Project Identity and Repository Evidence

The source tree is divided into `frontend`, `backend`, `ai-service`, `k8s`, `.github`, and several project-analysis documents. The backend package is named `backend`, and its entry point is `server.js`. The frontend package is a private Vite application. The AI service README calls the Python service an “AI Performance Analyzer” and mentions the PerTrack MERN app. The Kubernetes and Docker image names use `pfe`, which is consistent with a graduation-project context.

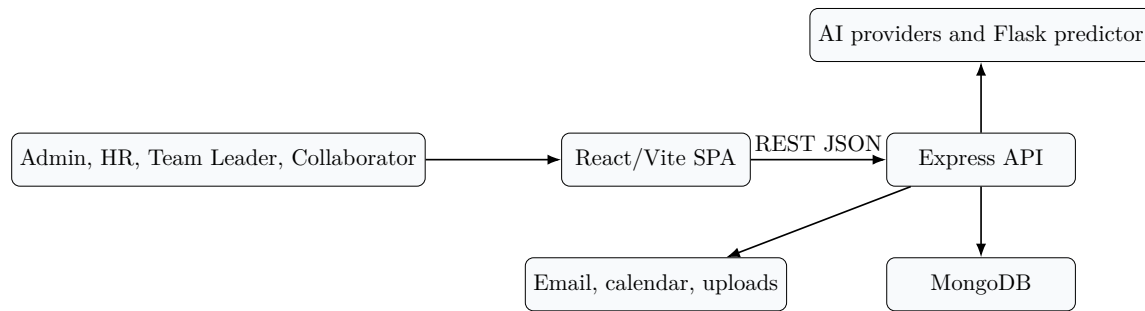


Figure 1.1: System-context diagram. Source: author’s own work based on repository architecture.

figure 1.1 separates the human actors from the software boundaries. The React application is the only direct interface for the four roles. The Express API centralizes authentication, authorization, validation, and business rules before touching MongoDB or external services. This separation matters because performance data should not be interpreted directly by the interface without backend checks.

1.5 Development Approach

The repository does not contain Jira, Trello, or GitHub Projects exports. It does contain a GitHub Actions workflow, test playbooks, audit documents, and several design diagrams. From code evidence, the development approach was iterative: modules were added around progressively deeper workflows such as objectives, team weight capacity, mid-year reviews, final evaluations, HR validation, AI support, and deployment automation. The presence of test files for bugs such as objective visibility, task privacy, and scoring rules also shows that the project went through corrective cycles rather than one single static implementation.

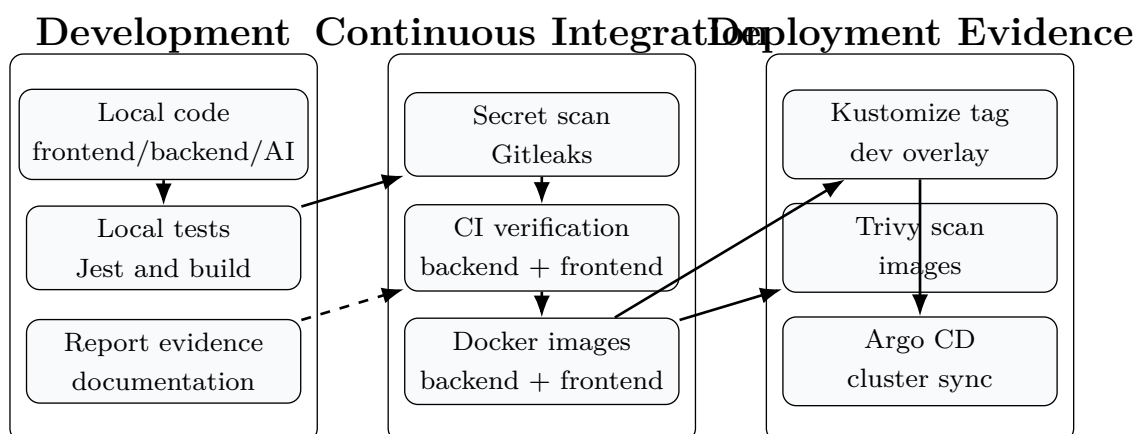


Figure 1.2: Repository workflow from local implementation to CI/CD evidence. Source: author’s own work based on repository folders and GitHub Actions.

figure 1.2 summarizes the engineering workflow that can actually be verified. There is no evidence of an external project-management board, so the report does not invent

one. The repository itself provides the stronger evidence: tests, CI stages, Docker images, Kustomize updates, and documentation artifacts.

1.6 Version Control and Quality Culture

Git is present in the repository, and CI is defined under `.github/workflows/docker-build.yml`. The workflow runs on pushes to `main`, `develop`, `qa`, and `staging`, as well as pull requests. It starts with secret scanning, then backend tests, frontend build and tests, Docker image build and push, image vulnerability scanning, Kustomize tag updates, and a smoke test for the development environment. This is a serious quality signal for a university project because it connects code changes to automated verification and deployment artifacts.

1.7 Chapter Conclusion

The verified project context is an HR performance management system implemented as a full-stack engineering project. Missing academic metadata is separated from technical evidence. The next chapter analyzes the business problem, expected solution, risks, and planning.

Chapter 2

Project Analysis, Existing Solutions, and Planning

2.1 Problem Statement

Manual performance management usually fails in three places. First, the objective agreement is often weakly documented, so employees and managers later debate what was expected. Second, progress evidence is fragmented, making mid-year and end-year reviews heavily dependent on memory. Third, HR validation can become a late administrative signature rather than a real process check. PerTrack targets these weaknesses by storing objectives, KPI targets, check-ins, tasks, review drafts, manager decisions, HR validation status, and audit logs in one system.

2.2 Project Objectives

The repository supports the following objectives:

- Provide authenticated access for four roles with route-level and UI-level restrictions.
- Model annual cycles with three phases: objective planning, mid-year review, and final evaluation.
- Support objective creation, approval, revision, acknowledgment, KPI management, comments, attachments, and progress updates.
- Track tasks, meetings, feedback, notifications, audit events, career actions, improvement plans, and HR decisions.
- Calculate objective-based performance scores with manager override and HR validation.
- Offer AI-assisted drafts and predictive analysis while keeping fallback paths.

- Package and deploy the system through Docker, Kubernetes, Kustomize, Argo CD, and GitHub Actions.

2.3 Scope

The implemented scope is an internal web platform rather than a public SaaS product. The `User` model enforces email addresses ending in `@biat.com`, which suggests an organization-specific deployment. Multi-tenancy appears only as a default `tenantId` field and is not implemented as a full isolation model. Calendar integration is present, but the repository does not prove a production OAuth configuration. AI generation depends on external API keys and uses fallback behavior when unavailable.

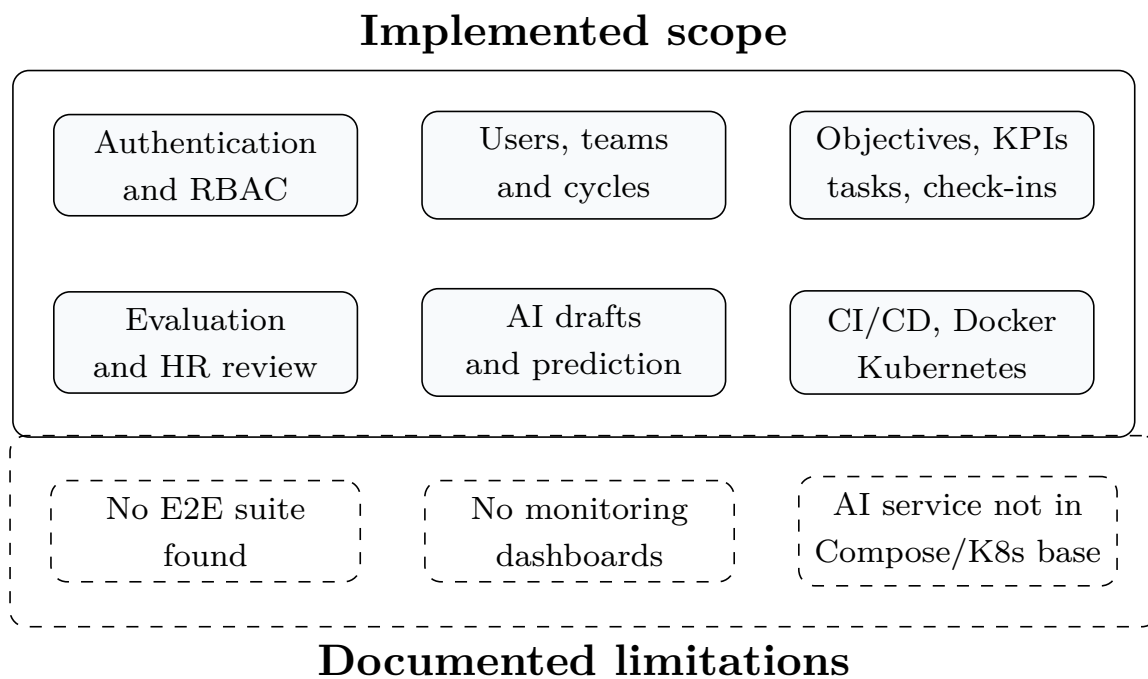


Figure 2.1: Project scope boundary and verified limitations. Source: author’s own work based on repository evidence.

[figure 2.1](#) is intentionally explicit about the boundary between delivered work and missing operational evidence. This avoids the common final-report problem of describing a prototype as if it were already a fully operated enterprise platform.

2.4 Comparable Solutions

Mature HR platforms generally include goal tracking, performance review forms, continuous feedback, reporting, and workflow approvals. PerTrack is narrower than a commercial HRIS, but it is deeper than a simple goal tracker because it includes phase governance,

manager and HR validation, score calculation, audit logs, task evidence, and AI support. The comparison below is conceptual and based on categories, not a claim about exact commercial product internals.

Table 2.1: Comparative analysis by feature category.

Category	Generic spreadsheet process	Generic HR platform	PerTrack implementation
Objective governance	Manual and hard to audit	Usually workflow-based	Explicit objective statuses, manager approval, revision, activity logs
KPI tracking	Often inconsistent	Often configurable	Embedded KPIs with metric type, current/target values, and status
Role permissions	Informal sharing	Product-specific RBAC	Backend role middleware plus frontend route restrictions
HR validation	Email or manual sign-off	Common in enterprise tools	Final evaluation status <code>pending_hr</code> , validation, return reason, workflow history
AI assistance	Not available	Increasingly available	LLM drafts plus Flask prediction service; synthetic data limitation stated
Deployment	Local documents	Vendor-hosted	Docker, Kubernetes, GitHub Actions, Argo CD manifests

2.5 Risk Analysis

Table 2.2: Project risk matrix.

Risk	Cause	Impact	Mitigation in the project
Incorrect score calculation	Mixed individual, team, and legacy objectives	Unfair final evaluation	Dedicated score service and Jest tests for weighting and normalization
Objective visibility leak	Managers and employees share sensitive drafts	Confidentiality issue	Visibility helper hides private draft individual objectives from non-owners
Phase inconsistency	Cycle moves before objectives are ready	Broken annual workflow	Phase checks in cycle routes before phase advancement
AI hallucination	LLM generates unsupported content	Misleading HR text	Prompts instruct use of supplied data only, JSON validation, fallback output
Synthetic model bias	Prediction service trained on generated data	Limited real-world validity	Report states limitation; AI service validates inputs and exposes confidence/probability
Secret exposure	CI/CD and AI keys require secrets	Security incident	Gitleaks secret scan, environment-variable validation, no secrets in report
Deployment drift	Kubernetes manifests diverge from image versions	Broken dev environment	GitOps tag update and Argo CD automated sync

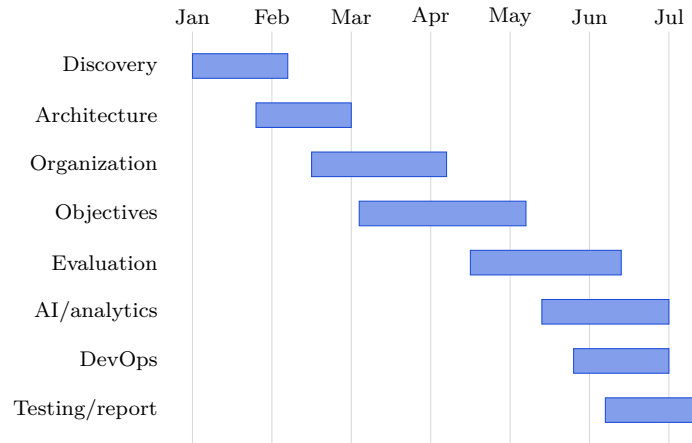


Figure 2.2: Indicative project planning reconstructed from repository history and module maturity. Source: author’s own work; exact internship dates are not present in the repository.

The Gantt chart is a reporting reconstruction, not an exported project-management schedule. It organizes the report around dependency order: identity and cycles first, objectives and work evidence second, evaluation and HR control third, AI and analytics fourth, and delivery quality last.

2.6 Planning by Sprints

The repository does not contain an official sprint calendar. For report clarity, implementation is organized into five engineering sprints reconstructed from module dependencies:

1. Foundation, organization, authentication, users, teams, cycles, and access control.
2. Objectives, KPIs, task management, check-ins, and phase-aware progress.
3. Evaluation workflows, final evaluation, HR validation, follow-up actions, and PDF/report exports.
4. AI assistance, prediction service, analytics, and auditability.
5. Testing, CI/CD, Docker, Kubernetes, GitOps, and deployment verification.

2.7 Chapter Conclusion

The project scope is coherent: it is an internal performance-management platform with strong workflow and governance needs. The main risks are not exotic technology problems; they are business-rule correctness, privacy, phase consistency, and honest AI use.

Chapter 3

Requirements and System Architecture

3.1 Functional Requirements

Table 3.1: Functional requirements verified from source code.

ID	Requirement	Evidence
FR-01	Authenticate users with access and refresh tokens	<code>backend/routes/auth.js</code> , <code>frontend/src/components/AuthContext.jsx</code>
FR-02	Restrict actions by role	<code>backend/middleware/role.js</code> , <code>frontend/src/routes/routeConfig.jsx</code>
FR-03	Manage users and profile avatars	<code>backend/routes/users.js</code> , <code>User.js</code>
FR-04	Manage teams and subteams	<code>backend/routes/teams.js</code> , <code>Team.js</code>
FR-05	Create and advance performance cycles	<code>backend/routes/cycles.js</code> , <code>Cycle.js</code>
FR-06	Create, submit, approve, revise, evaluate, and lock objectives	<code>backend/routes/objectives.js</code> , <code>objectiveController.js</code>
FR-07	Track KPIs embedded inside objectives	<code>Objective.js</code> , objective KPI routes
FR-08	Submit and review check-ins with attachments	<code>CheckIn.js</code> , <code>checkInController.js</code>
FR-09	Manage kanban-like tasks and time tracking	<code>Task.js</code> , <code>frontend/src/pages/TasksPage.jsx</code>
FR-10	Generate final evaluations and HR validations	<code>FinalEvaluation.js</code> , <code>finalEvaluationController.js</code>

ID	Requirement	Evidence
FR-11	Record HR decisions, improvement plans, and bonus/penalty decisions	<code>HRDecision.js</code> , <code>ImprovementPlan.js</code> , <code>BonusPenalty.js</code>
FR-12	Provide AI drafts and predictions	<code>backend/routes/ai.js</code> , <code>ai-service/app.py</code>
FR-13	Provide analytics, reports, audit logs, and notifications	<code>stats.js</code> , <code>reports.js</code> , <code>auditLog.js</code> , <code>notifications.js</code>

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements and implementation evidence.

Requirement	Implementation	Limitation
Security	Helmet, CORS allowlist, rate limiting, XSS cleaning, Mongo sanitization, JWT, bcrypt, RBAC	No formal penetration test or CSRF protection evidence
Traceability	Audit log model, audit helpers, objective activity logs, final evaluation workflow history	Audit coverage may not include every route
Maintainability	Route-controller-model organization, services for scoring and AI, shared frontend utilities	Some legacy duplicated evaluation modules remain
Usability	Role-specific routes, dashboard shell, empty/loading components, charts, modals	No accessibility audit found
Portability	Dockerfiles, Docker Compose, Kubernetes base and overlays	AI service is not included in Compose/Kubernetes base manifests
Testability	Jest, Supertest, React Testing Library, validation and scoring tests	No coverage percentage stored in repository

3.3 Architecture

The frontend and backend communicate through REST-style JSON endpoints. React Router defines the protected pages, while Axios clients attach access tokens and refresh them when a request returns 401. The backend authenticates JWT bearer tokens, loads the current user profile, checks role permissions, validates request payloads with Joi in selected routes, and stores data through Mongoose.

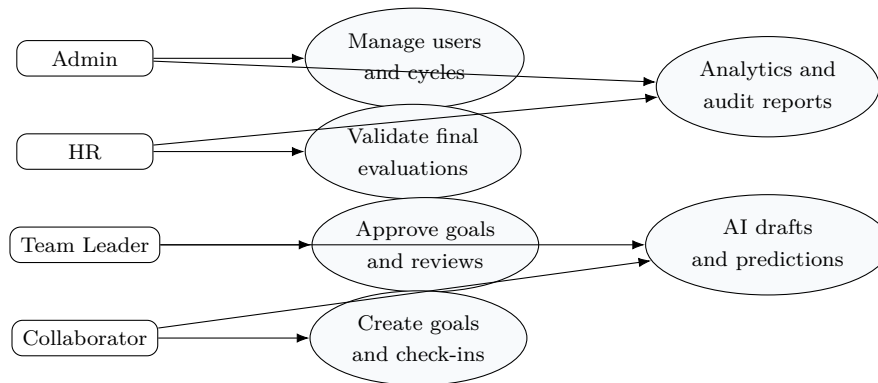


Figure 3.1: Global use-case diagram. Source: author’s own work based on routes and frontend route configuration.

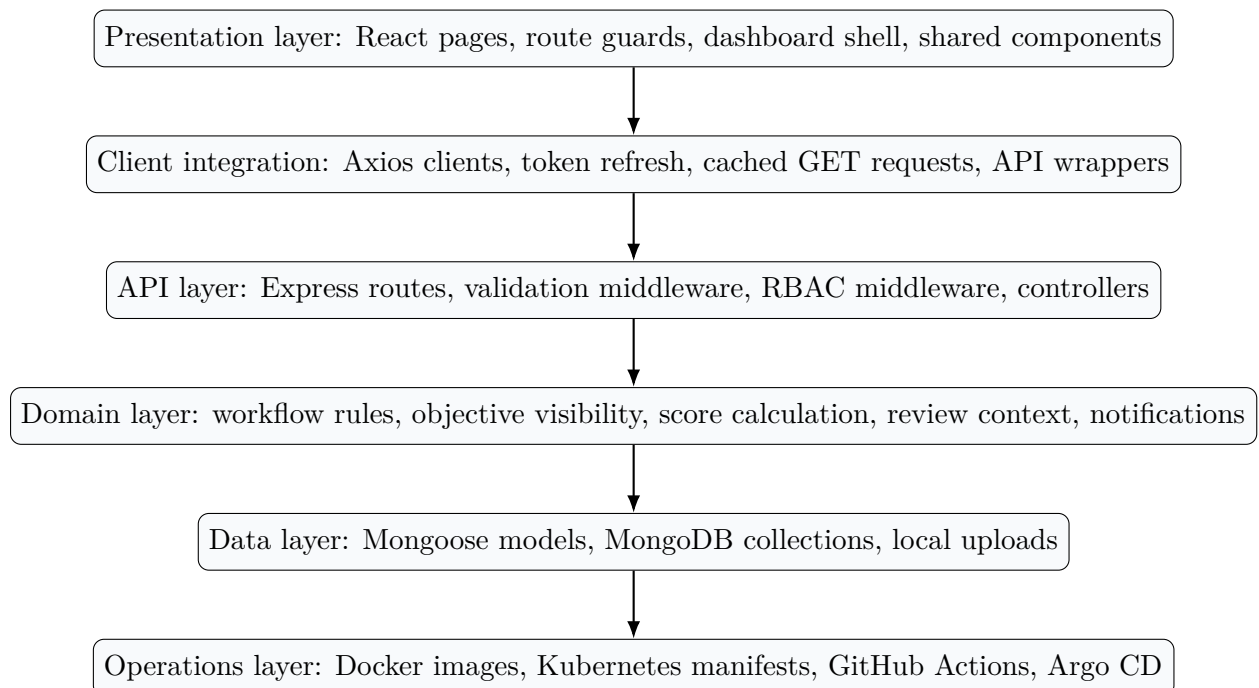


Figure 3.2: Logical architecture. Source: author’s own work based on application source code.

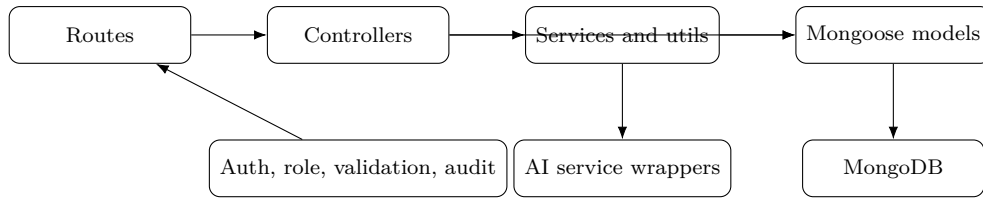


Figure 3.3: Backend component diagram. Source: author’s own work based on Express route and controller structure.

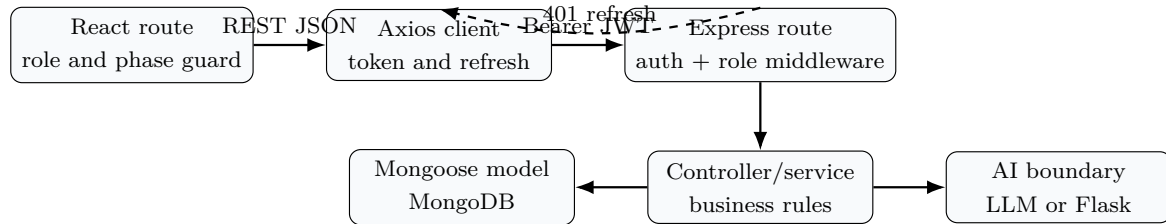


Figure 3.4: Frontend, backend, database, and AI communication flow. Source: author’s own work based on API client, routes, services, and AI files.

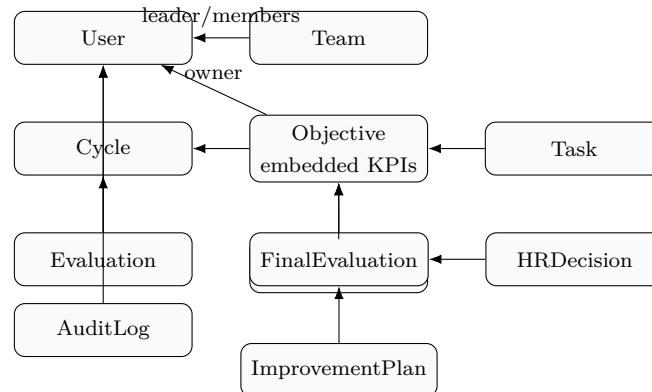


Figure 3.5: Database model overview. Source: author’s own work based on Mongoose schemas.

Together, figures 3.1 and 3.4 show the two sides of the architecture. The use-case view explains what each actor can do, while the communication view shows how a protected frontend action becomes an authenticated backend request and, when needed, a database or AI-service interaction.

3.4 Technology Choices

React is used for component-driven UI construction [3]. Vite provides the development and production build workflow [4]. Express provides routing and middleware composition [5]. Mongoose defines MongoDB schemas, validation, hooks, indexes, and references [2].

Flask is used for the Python prediction API through route decorators and JSON request handling [6]. XGBoost and scikit-learn provide the implemented model APIs [7], [8].

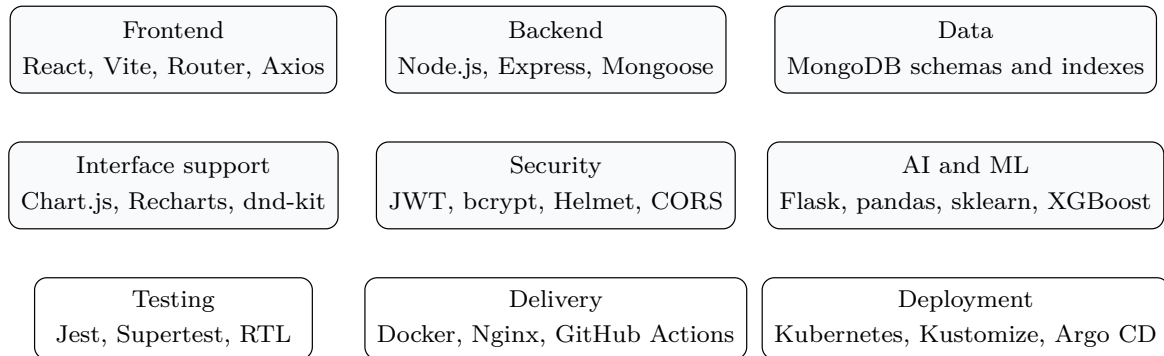


Figure 3.6: Verified technology stack grouped by project responsibility. Source: author’s own work based on manifests and deployment files.

figure 3.6 groups technologies by responsibility rather than listing them as isolated definitions. The trade-off is visible: the application gains separation of concerns, but it also requires careful coordination between browser state, backend authorization, MongoDB schema evolution, and a separate AI runtime.

Table 3.3: Technology-choice table based on package manifests.

Layer	Verified technology	Role in project
Frontend	React 18, Vite, React Router, Axios	SPA, routing, API communication, role-aware pages
UI analytics	Chart.js, Recharts, react-chartjs-2	Dashboards and performance visualizations
Backend	Node.js, Express 4, Mongoose 7	REST API, middleware, persistence, domain rules
Security	JWT, bcryptjs, Helmet, CORS, rate limit, xss-clean, mongo-sanitize	Authentication and request hardening
AI generation	OpenAI SDK, Google Generative AI package	LLM provider abstraction for review drafts and suggestions
AI prediction	Flask, pandas, scikit-learn, XGBoost, joblib	Prediction API and saved model artifacts
DevOps	Docker, Nginx, Kubernetes, Kustomize, Argo CD, GitHub Actions	Build, deploy, GitOps and CI/CD

3.5 API Architecture

The backend mounts 26 route modules under `/api`. The main API groups are authentication, users, team members, teams, cycles, objectives, HR decisions, notifications, feed, statistics, audit logs, meetings, AI, feedback, tasks, calendar, career, performance, reports, evaluations, PDF export, check-ins, final evaluations, improvement plans, and bonus/penalty.

3.6 Chapter Conclusion

The architecture is a practical MERN-style application with a separate Python AI service. The strongest architectural feature is the separation of domain rules into services and utilities; the main architectural limitation is that the AI prediction microservice is not fully integrated into the Docker/Kubernetes deployment files.

Chapter 4

Sprint 1: Organization, Access Control, and Cycle Foundation

4.1 Sprint Objective

The first implementation sprint establishes the foundation required by all later performance workflows: authentication, users, roles, teams, subteams, current-user context, protected frontend routes, and performance cycles.

4.2 Authentication

Login is implemented in `backend/routes/auth.js`. The backend verifies email and password, compares the submitted password with the bcrypt hash stored in MongoDB, and returns an access token plus a refresh token. The frontend stores both tokens in `localStorage`. `AuthContext.jsx` keeps the current user in React context, persists a lightweight user object, and refreshes the access token through `/api/auth/refresh` when necessary.

JWT is a compact claims format standardized in RFC 7519 [9]. In this application, the token is used for stateless API authentication, while the refresh token is also stored against the user record. The implementation validates that a token maps to an active, non-deleted user before setting `req.user`. This extra lookup prevents deleted or disabled users from continuing to access the API only because they still hold an unexpired token.

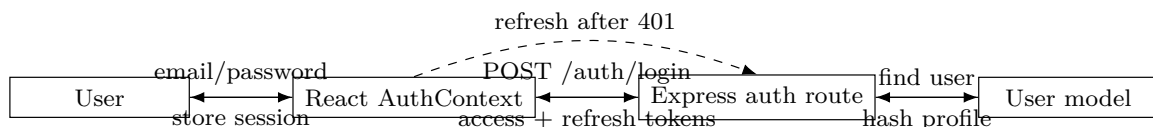


Figure 4.1: Authentication and token-refresh sequence. Source: author’s own work based on ‘AuthContext.jsx’, ‘api.js’, and backend auth middleware.

figure 4.1 highlights a small but important engineering decision: the backend does not trust the token alone. It loads the user profile and checks that the account is still active. That adds a database lookup, but it prevents a common administrative problem where a disabled user keeps access until token expiry.

4.3 Role-Based Access Control

The role list is defined in `backend/models/User.js`: `ADMIN`, `HR`, `TEAM_LEADER`, and `COLLABORATOR`. The `role.js` middleware rejects requests where the current role is not in the allowed set, while automatically allowing the administrator to pass role checks. Frontend route metadata repeats role restrictions so the navigation shell does not advertise pages the current user should not access.

Table 4.1: Role-permission matrix. Source: author’s own work based on backend role middleware and route configuration.

Capability	Admin	HR	Team leader	Collaborator
User administration	Full	Limited lists	Managers/collaborators list	Own profile
Cycle phase control	Full	View	View	View
Team creation	Full	Create/update	Subteam management	View own team
Objective drafting	Yes	Limited	Yes	Yes
Objective approval	Yes	No direct route	Managed team	No
Final evaluation	Yes	Validate/review	Generate/update	Feedback/own history
Audit logs	Yes	Yes	Entity history only	No
Bonus/penalty	Yes	Review	Propose/create route access	View scoped

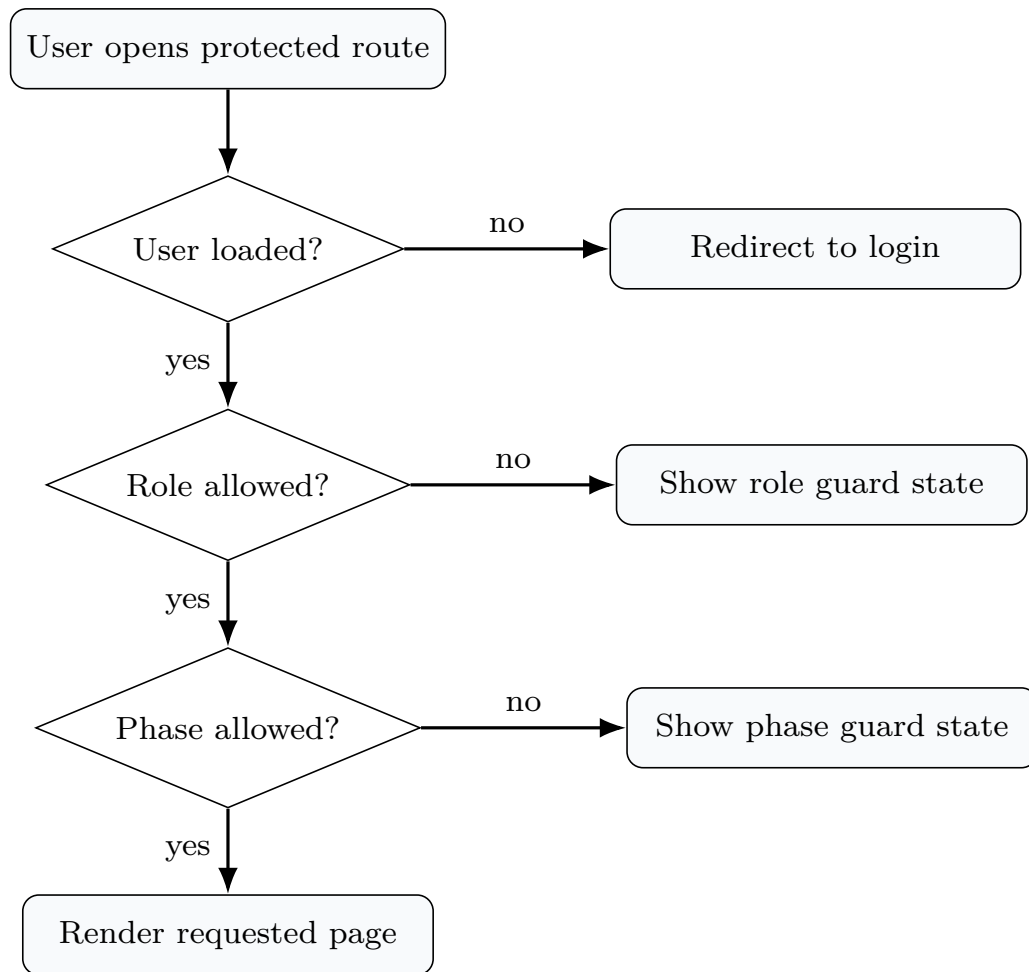


Figure 4.2: Frontend route-guard decision flow. Source: author’s own work based on ‘RouteGuard.jsx’ and route metadata.

[figure 4.2](#) shows the frontend side of the same control. The interface can hide or block pages based on role and cycle phase, but this is not treated as the only security layer. Backend route middleware remains responsible for enforcing the API boundary.

4.4 Team and Subteam Management

The **Team** schema stores a name, description, leader, members, parent team, and creator. This supports both main teams and subteams through the **parentTeam** reference. Route code checks whether a team leader manages a root team or one of its descendants. That recursive managed-team logic appears in **backend/utils/accessControl.js** and is reused by employee access decisions.

Team creation validates role coherence: leaders must be **TEAM_LEADER**, **ADMIN**, or **HR**; members must be collaborators; and a manager cannot be assigned to multiple root teams in the same way. This reduces administrative ambiguity and avoids a common problem in HR tools: a person appears under two managers with conflicting evaluation authority.

4.5 Cycle Management

Performance cycles are stored in `Cycle.js`. Each cycle has a unique year, status, and six dates for phase starts and ends. The phase structure is explicit: `phase1`, `phase2`, `phase3`, and `closed`. The schema validates sequential phase dates when they are modified. The routes add business checks before phase advancement, including checks for unapproved objectives and incomplete readiness conditions.

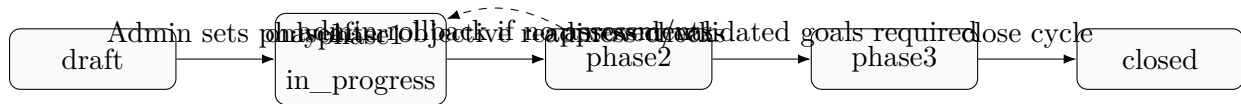


Figure 4.3: Performance-cycle state diagram. Source: author’s own work based on `backend/routes/cycles.js`.

4.6 Security Controls Introduced in This Sprint

The Express app applies Helmet, CORS, compression, JSON parsing, XSS cleaning, MongoDB sanitization, cache-control headers, rate limiting, and protected static-upload rules. Passwords are hashed with bcrypt using 12 salt rounds. The implementation also hides check-in upload files behind a blocked static route and serves other uploads with caching.

These controls reduce common web risks but do not make the system fully secure by themselves. OWASP API guidance emphasizes that authorization mistakes at object level and property level remain common API risks [10]. PerTrack addresses some of this through helper functions such as `canAccessEmployee`, `canAccessTeam`, and `canAccessObjective`, but a complete security assessment would still require endpoint-by-endpoint review and dynamic testing.

4.7 Sprint Result

At the end of this sprint, the application can identify users, enforce broad role permissions, represent teams and subteams, create annual cycles, and expose protected frontend pages. These foundations make later workflows possible because every objective, task, check-in, and evaluation needs a user, role, team scope, and active cycle.

4.8 Design Decisions and Trade-offs

The main trade-off in this sprint is between convenience and control. Storing tokens in browser storage simplifies the React session model, but it increases the importance of XSS prevention and short-lived access tokens. The backend partly compensates by

checking the user record on authenticated requests and by applying request-hardening middleware. Another trade-off appears in role duplication: route roles exist in both frontend metadata and backend middleware. This duplication is acceptable here because the frontend improves navigation clarity while the backend remains the authority.

Chapter 5

Sprint 2: Objectives, KPIs, Tasks, and Check-Ins

5.1 Sprint Objective

The second sprint implements the work-management core of the system. Objectives define expected performance, KPIs make success measurable, tasks create day-to-day execution evidence, and check-ins provide periodic progress records.

5.2 Objective Model

`Objective.js` is one of the richest schemas in the repository. It stores title, due date, description, success indicator, owner, cycle, category, team, assigned users, weight, priority, achievement percent, self-assessment fields, final self-assessment fields, manager adjustment, weighted score, workflow status, source, assignment metadata, rejection and revision reasons, evaluation fields, labels, visibility, parent objective, embedded KPIs, progress updates, comments, attachments, change requests, activity logs, and manager notes.

5.3 Objective Lifecycle

The implemented status list includes `draft`, `pending`, `submitted`, `pending_approval`, `revision_requested`, `rejected`, `assigned`, `acknowledged`, `approved`, `validated`, `locked`, `cancelled`, `evaluated`, and `archived`. The list preserves legacy states while supporting newer states such as manager-assigned objectives and correction requests.

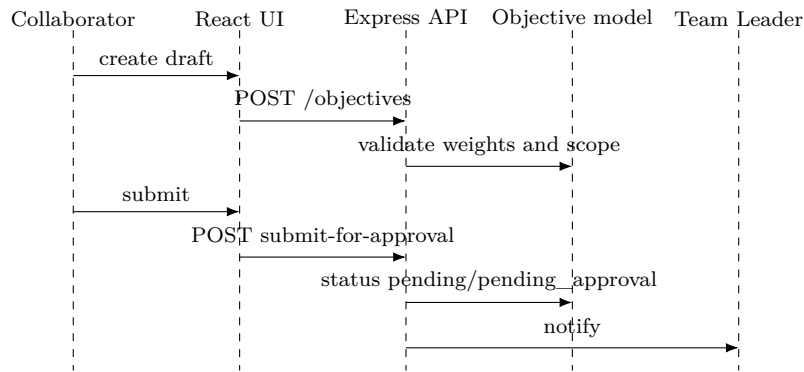


Figure 5.1: Objective-submission sequence diagram. Source: author’s own work based on objective routes and controller workflow.

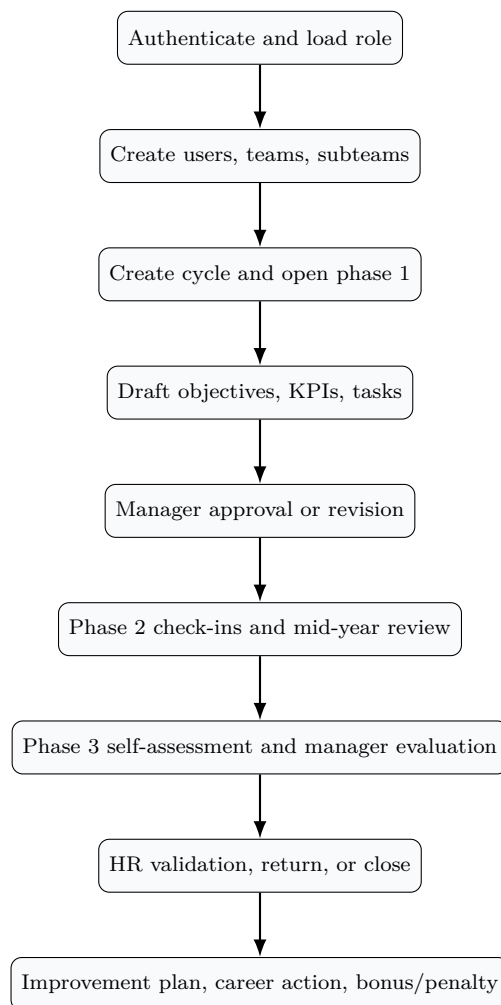


Figure 5.2: Global workflow activity diagram. Source: author’s own work based on implemented modules.

5.4 Weight Rules

Objective weights are central business rules. The code validates individual objective weights between 1 and 100, calculates totals, excludes cancelled and archived objectives from allocation, deduplicates team objective copies, and computes employee allocation in separate buckets for individual, team, and subteam objectives.

For one objective, the weighted contribution is:

$$P_o = \frac{w_o \times a_o}{100}$$

where w_o is the objective weight and a_o is the achievement percentage used for scoring.

For final evaluation scoring, eligible objective statuses are **approved**, **validated**, **evaluated**, and **locked**. If the total eligible weight is exactly 100, the score is the sum of weighted points. If the eligible total is not 100, the score is normalized:

$$S = \begin{cases} \sum P_o, & \text{if } \sum w_o = 100 \\ \frac{\sum P_o}{\sum w_o} \times 100, & \text{if } 0 < \sum w_o \neq 100 \\ 0, & \text{if } \sum w_o = 0 \end{cases}$$

This normalization exists to avoid unfairly penalizing legacy or incomplete cycles where the administrative total does not reach 100 percent.

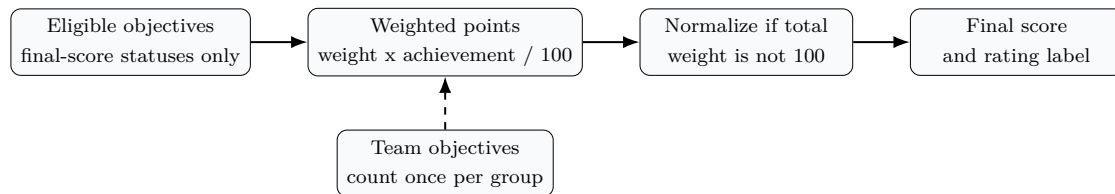


Figure 5.3: Objective scoring and weight-normalization logic. Source: author’s own work based on scoring and objective-rule utilities.

figure 5.3 captures the rule that required the most care in this sprint. A scoring function must not blindly average progress values, because a 10 percent objective and a 40 percent objective should not influence the final score equally. It must also avoid double-counting distributed team objectives.

Listing 5.1: Core weighted-points rule from the scoring service.

```
const weightedPoints = Number(((weight * achievement) / 100).toFixed(2));
```

5.5 Team Objective Weighting

The tests make a subtle rule explicit: a team objective weight is not divided among members. If a 40 percent team objective is assigned to four members, it consumes 40 percent of each assigned member’s team-objective budget. This is important because it preserves the importance of shared work instead of making it vanish as team size grows.

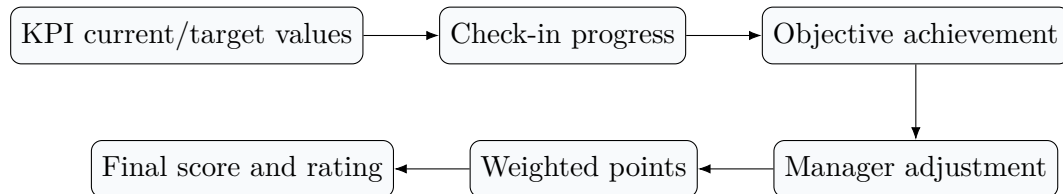


Figure 5.4: KPI and scoring flow. Source: author’s own work based on objective, check-in, and scoring modules.

5.6 KPI Management

KPIs are embedded inside objectives. Each KPI has title, metric type, initial value, target value, current value, unit, and status. Supported metric types are percent, number, currency, boolean, and milestone. The frontend detail panel lets users add KPIs, update current values, search KPIs, and visualize progress. The backend exposes routes to add, update, and delete KPIs under `/api/objectives/:id/kpis`.

5.7 Tasks

The `Task` model supports title, description, assignee, assignedBy, status, priority, work-flow stage, progress, labels, due date, completion date, recurrence, linked goal, phase, KPI id, linked meeting, team, notes, and time tracking sessions. The frontend `TasksPage.jsx` implements kanban-like behavior using statuses such as `todo`, `in_progress`, `done`, and `cancelled`. Time tracking is stored both in a nested `timeTracking` structure and legacy-style `timeSessions`, showing compatibility with evolving frontend features.

5.8 Check-Ins

Check-ins connect an employee, objective, and cycle. They include progress percentage, notes, priority, attachments, status, history, manager feedback, and review metadata. A manager or authorized HR/admin actor can approve a check-in or request revision. When a reviewed check-in includes progress, the controller may update the related objective’s achievement percentage. This creates a data path from periodic evidence to objective progress.

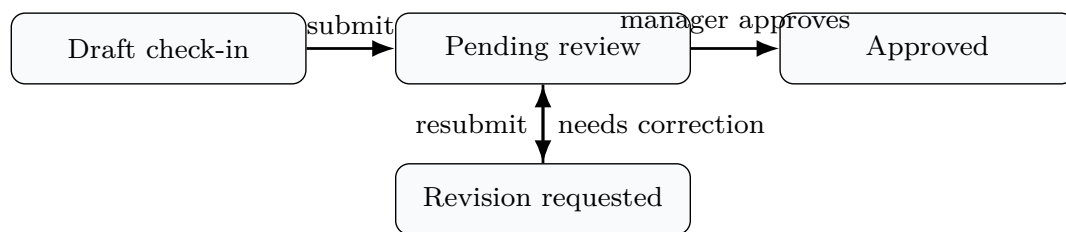


Figure 5.5: Check-in review workflow. Source: author’s own work based on ‘CheckIn.js’ and check-in routes.

UI screenshots for the objectives and task workspaces are deferred in this revision by request. The implemented views remain documented through route evidence, frontend module inventory, and the workflow diagrams above.

5.9 Sprint Result

This sprint turns performance planning into concrete work data. Objectives hold the contract, KPIs hold the measurement structure, tasks hold execution evidence, and check-ins hold periodic progress. The largest engineering challenge is consistency: the same objective must behave correctly across role visibility, status changes, team distribution, weight budgets, and final scoring.

5.10 Implementation Challenges

The first challenge is objective visibility. Employee-created draft individual objectives should stay private until submission, while assigned or approved objectives must become visible to the responsible manager. This is why the repository contains a dedicated visibility helper instead of scattering the rule across controllers. The second challenge is weight capacity. Team objectives cannot be divided by the number of members without weakening shared goals, but counting the full weight for every member means the backend has to protect capacity carefully. The tests around objective rules and score calculation show that these edge cases were treated as business rules, not presentation details.

Chapter 6

Sprint 3: Evaluation, HR Review, and Follow-Up Decisions

6.1 Sprint Objective

The third sprint implements the evaluation and HR governance workflows. It transforms approved objectives and progress evidence into manager reviews, HR-validated final evaluations, follow-up plans, career recommendations, and compensation or discipline-related decisions.

6.2 Two Evaluation Layers

The repository contains both `Evaluation` and `FinalEvaluation`. The first model represents a general manager evaluation workflow with statuses from `draft` to `completed`, suggested score, final score, score history, approvals, and employee acknowledgment. The second model is richer and appears to be the main end-year workflow: it stores objective breakdown, evidence summaries, manager score, final score, rating label, HR status, workflow history, performance status, recommendation, HR decision, and employee feedback.

This distinction matters. A report that merges both models into one generic evaluation feature would hide the real implementation. The application has a general evaluation module and a final annual evaluation module, with overlapping but different responsibilities.

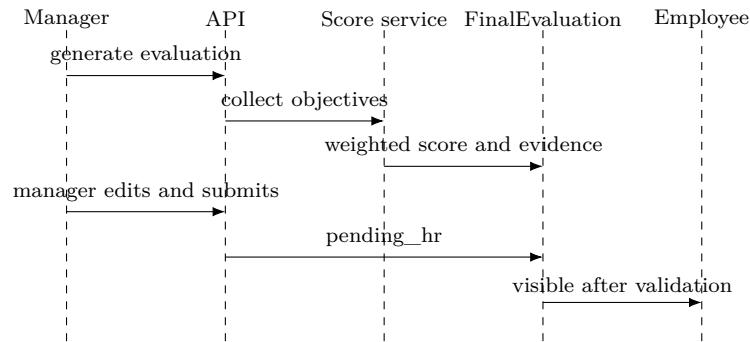


Figure 6.1: Final evaluation sequence diagram. Source: author’s own work based on final evaluation controller and scoring service.

6.3 Automatic Score and Manager Adjustment

`scoreCalculationService.js` calculates automatic objective-based performance. It chooses the achievement value in priority order: manager-adjusted percent, final self percent, then current achievement percent. This means manager confirmation can override employee input, while still preserving the employee achievement in the objective breakdown.

Rating labels are mapped from final score:

Table 6.1: Final score rating bands implemented in `scoreCalculationService.js`.

Score range	Rating label
90–100	exceptional
75–89	strong
50–74	meets_expectations
30–49	needs_improvement
0–29	unsatisfactory

6.4 HR Validation

HR validation is more than changing a status. `workflowRules.js` builds warnings and blocking issues before HR review. It checks whether final score, rating, manager comments, strengths, weaknesses, objective breakdown, AI review confirmation, improvement plan, career recommendation, and bonus documentation are coherent. Some warnings are advisory, while blocking issues prevent validation.

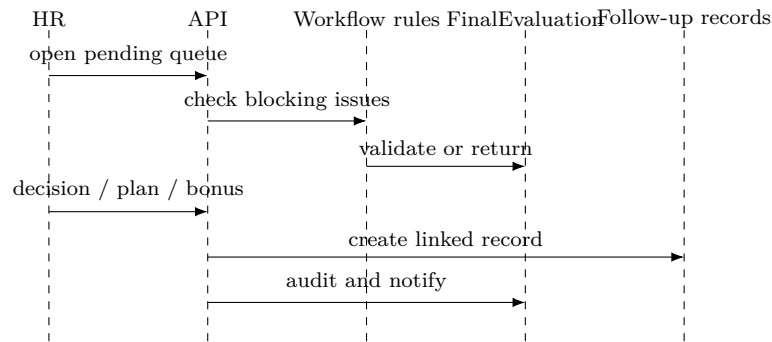


Figure 6.2: HR-review sequence diagram. Source: author’s own work based on HR validation, decision, and improvement-plan modules.

Table 6.2: Examples of HR review controls implemented in workflow rules.

Control	Why it matters	Effect
Missing final score or rating	HR cannot validate an incomplete evaluation	Blocking issue
Rating inconsistent with final score	Prevents accidental label-score mismatch	Warning or issue depending on context
Large manager override without justification	Protects against unexplained score changes	Warning before validation
Low score without improvement plan	Connects poor performance to follow-up action	Warning
Bonus or penalty without reason/value	Prevents unsupported HR decision records	Blocking validation for that action

6.5 Improvement Plans

The `ImprovementPlan` model links a plan to a final evaluation, employee, and cycle. It stores the improvement objective, deadline, expected outcome, notes, progress status, creator, and updater. The controller restricts creation and destructive actions primarily to HR and admin actors, while allowing scoped visibility for the employee and team leader. This is a realistic design because improvement plans are sensitive and should not behave like ordinary tasks.

6.6 HR Decisions and Bonus/Penalty Records

`HRDecision` stores the employee, cycle, final evaluation, individual score, team score, final score, action, action label, decision maker, date, and notes. `BonusPenalty` stores a bonus or penalty record linked to an employee, final evaluation, optional HR decision,

and optional objective. It also stores approval status and review notes. The workflow rules validate that bonus or penalty value must be positive and that a reason is required.

6.7 Career Development

Career modules include competencies, career paths, development actions, and career recommendations. Recommendations can be generated from a final evaluation and saved with suggested path, skills to develop, source, and basis. The design links development decisions to performance evidence rather than isolating career management as a separate module.

6.8 PDF and Report Exports

The backend contains PDF routes using PDFKit for team and user reports, and final evaluation export routes. The frontend also includes PDF/export-related components and libraries such as `jspdf` and `html2canvas`. The repository verifies PDF capability, but it does not prove that every final report view was manually exported in a production environment.

Screenshots of the manager evaluation and HR validation pages are deferred in this revision by request. The workflow remains supported by model fields, route permissions, sequence diagrams, and the HR rule table above.

6.9 Sprint Result

The evaluation sprint gives the project its business value. It connects objective evidence with manager judgment and HR governance. The implementation is strongest where it validates process quality before HR validation. Its main limitation is complexity: two evaluation models coexist, and future maintenance should keep their responsibilities clearly separated or consolidate them deliberately.

6.10 Design Decisions and Trade-offs

Separating manager scoring from HR validation clarifies responsibility. The manager evaluates performance evidence; HR checks whether the process is complete, justified, and aligned with follow-up rules. The cost of this separation is workflow complexity: an evaluation can be generated, edited, recalculated, submitted, returned, validated, and later connected to decisions or improvement plans. The embedded workflow history in

FinalEvaluation helps control that complexity by preserving who acted and which state changed.

Chapter 7

Sprint 4: AI Assistance, Prediction, Analytics, and Auditability

7.1 Sprint Objective

The fourth sprint adds decision-support features: AI-assisted writing, performance prediction, dashboard analytics, reports, and audit logs. These modules are useful only if the report is careful about what is implemented and what remains a limitation.

7.2 Generative AI Service

The Node service `backend/services/aiService.js` is provider-agnostic. It supports Gemini through `@google/generative-ai`, and OpenAI-compatible chat completions for xAI/Grok and OpenAI through the `openai` package. Provider, model, timeout, and maximum input size are controlled by environment variables.

The service builds compact context strings, truncates overly large context, asks for JSON-only outputs, parses JSON, performs light cleanup if necessary, validates the returned fields, and falls back to empty structured output with a warning when generation fails. Review modes include mid-year summary, final self-assessment, manager review, and development plan. This is a practical pattern because AI outputs are treated as drafts, not as authoritative HR decisions.

7.3 AI Controller Capabilities

The backend exposes endpoints for goal suggestions, KPI suggestions, performance summaries, risk detection, notification prioritization, assistant responses, check-in drafts, objective-quality analysis, objective refinement, mid-year review, final self-review, manager review, development plan, evaluation draft generation, prediction user lists, em-

ployee performance predictions, and prediction requests. Permission checks are stronger for manager-review and evaluation-draft paths than for general assistance paths.

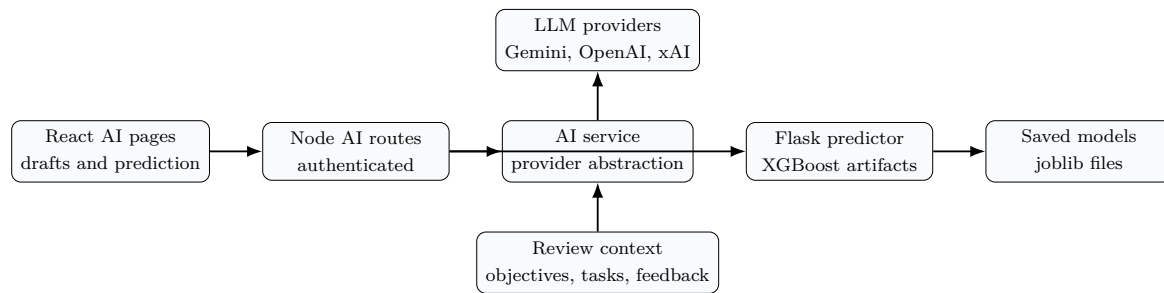


Figure 7.1: AI architecture separating generative drafting from predictive scoring. Source: author’s own work based on backend AI services and the Flask AI service.

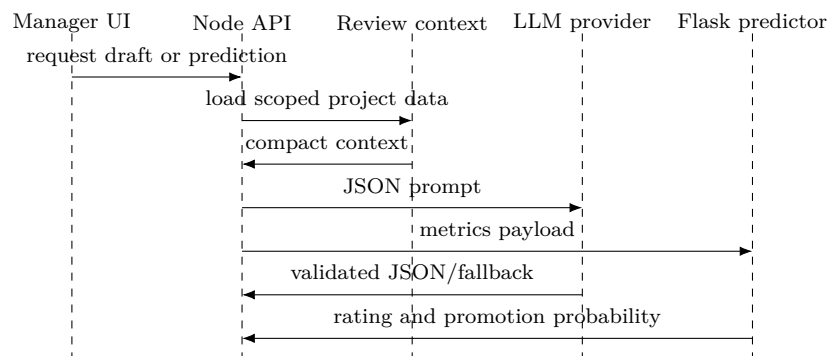


Figure 7.2: AI inference sequence diagram. Source: author’s own work based on Node AI service and Flask API.

[figure 7.1](#) separates two AI responsibilities that should not be confused. Generative drafting depends on a provider abstraction and review context gathered from the main application. Predictive scoring depends on the Python Flask service and saved model artifacts. Both are useful, but neither replaces HR validation.

7.4 Python Prediction Service

The `ai-service` folder contains a Flask API, a synthetic CSV dataset with 10,000 rows, training code, saved model artifacts, and a shared text generator. The prediction API exposes `GET /health`, `POST /predict`, and `POST /predict/batch`. The API requires eight numeric features:

- KPI score
- Goal completion percentage
- Check-in count
- Average check-in progress

- Feedback count
- Positive feedback ratio
- Task completion percentage
- Tasks-on-time percentage

The API validates bounds for every feature, loads `rating_xgb.joblib`, `promotion_xgb.joblib`, and `rating_label_encoder.joblib`, then returns rating, rating confidence, promotion readiness, promotion probability, deterministic overall score, strengths, weaknesses, summary, and suggestions.

7.5 Prediction Formula

The service computes an overall score independently of the model:

$$S_{\text{overall}} = 0.30 KPI + 0.25 Goal + 0.15 CheckInProgress \\ + 0.20 TaskCompletion + 0.10(PositiveFeedbackRatio \times 100)$$

The training script deliberately excludes this `overall_score` column from the model inputs because it is deterministic from the raw features. This avoids allowing the classifier to learn a trivial threshold from a precomputed score.

7.6 Dataset and Model Evidence

The included dataset has 10,000 rows, 0 missing values, and 0 duplicate rows. Rating distribution is: 2,238 exceptional, 2,722 exceeds expectations, 2,440 meets expectations, and 2,600 needs improvement. Promotion readiness is intentionally probabilistic by rating: about 79.62 percent for exceptional, 51.03 percent for exceeds expectations, 21.02 percent for meets expectations, and 2.46 percent for needs improvement.

Table 7.1: Model evaluation on the saved artifacts using the repository training split.

Task	Model	Metric	Result
Rating classification	Random Forest	Accuracy	0.9715
Rating classification	XGBoost	Accuracy	0.9780
Promotion readiness	Random Forest	Accuracy / ROC-AUC	0.7505 / 0.8346
Promotion readiness	XGBoost	Accuracy / ROC-AUC	0.7480 / 0.8218

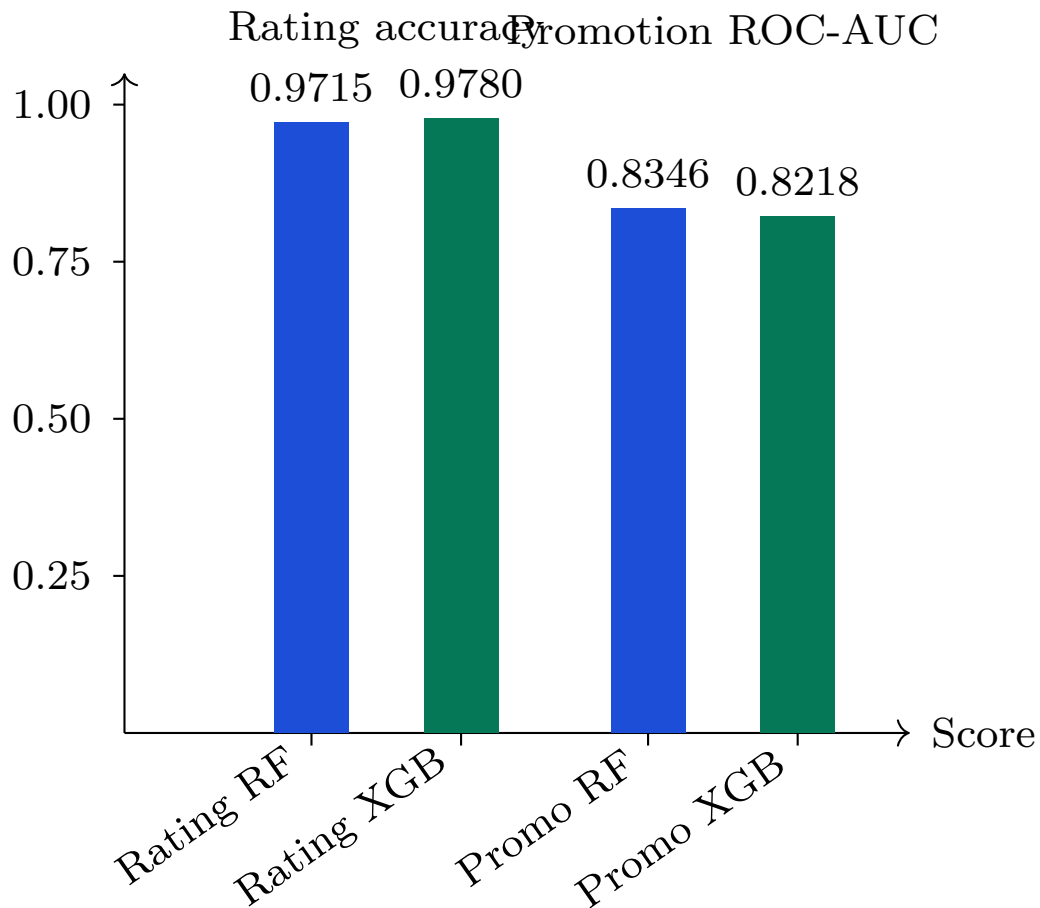


Figure 7.3: Model comparison from saved artifact evaluation. Source: project AI artifacts evaluated locally.

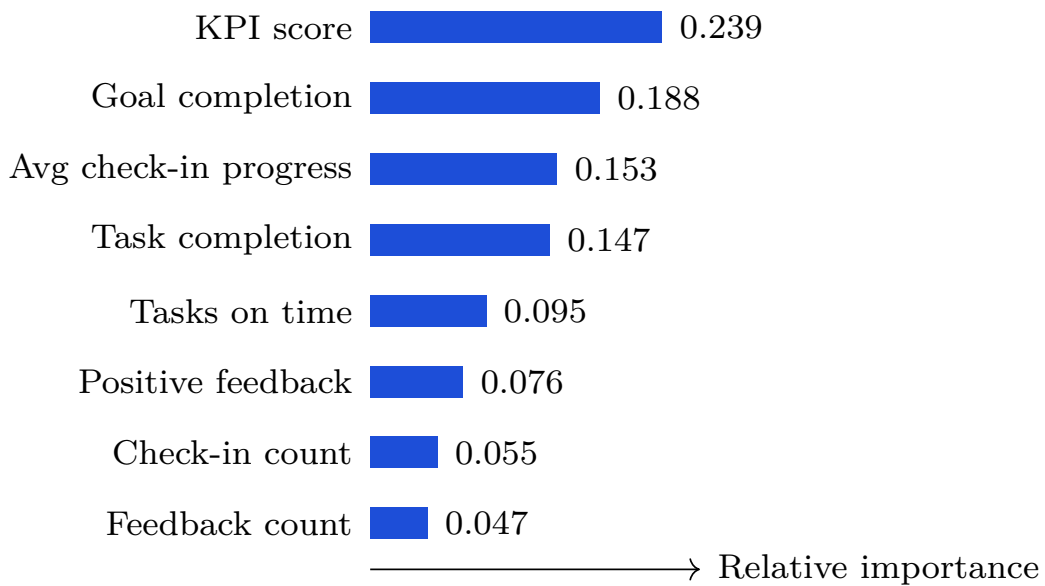


Figure 7.4: Illustrative feature-importance ranking from the saved XGBoost rating model evaluation. Source: project AI artifacts evaluated locally.

The selected API models are XGBoost for both tasks. This is defensible for the rating task because XGBoost has the strongest measured accuracy in the repository evaluation. For promotion readiness, Random Forest has slightly higher ROC-AUC in the local evaluation, while the API still uses XGBoost. This should be documented as an implementation choice rather than overstated as universally superior.

7.7 Analytics

Analytics endpoints exist under `/api/stats`, `/api/performance`, and `/api/reports`. The frontend includes dashboards, performance pages, analytics pages, prediction pages, and chart libraries. Dashboard utilities compute status summaries, completion rates, check-in summaries, task summaries, timeline buckets, and performance trend data.

7.8 Auditability

Auditability appears at two levels. First, `AuditLog` stores user, action, entity type, entity id, entity name, role, description, changes, metadata, IP address, and timestamp. Second, objectives and final evaluations keep embedded activity or workflow history. This combination is useful because system-wide audit logs answer compliance questions, while embedded history gives contextual traceability inside a business object.

7.9 AI Limitations and Ethics

The AI prediction dataset is synthetic. Therefore, prediction metrics cannot be presented as real organizational performance validity. A real deployment would need historical data governance, consent and privacy review, bias analysis, explainability review, retraining controls, and monitoring for drift. The generative AI system can draft text, but the project already recognizes the need for human review through warnings such as `AI-assisted draft has not been confirmed as reviewed by the manager`. That is the correct posture for HR use.

Prediction and analytics UI screenshots are deferred in this revision by request. The AI evidence therefore relies on repository artifacts, API behavior, saved model metrics, and the charts generated from those artifacts.

7.10 Sprint Result

The AI and analytics sprint adds value by helping users write better reviews and interpret performance evidence. Its most important engineering quality is that the code contains

validation and fallback paths. Its most important limitation is also clear: predictive AI is trained on synthetic data and should not be treated as a production HR decision engine without real validation.

7.11 Design Decisions and Trade-offs

The project makes a sensible split between deterministic scoring and predictive assistance. The deterministic score is explainable and can be reviewed objective by objective. The ML prediction is exploratory and should support discussion, not decide outcomes. Keeping both paths separate avoids a dangerous shortcut where a model prediction silently becomes the official evaluation score.

Chapter 8

Sprint 5: Testing, Continuous Integration, and Deployment

8.1 Sprint Objective

The fifth sprint verifies the application and prepares it for repeatable deployment. The repository includes local tests, production frontend build scripts, Dockerfiles, Docker Compose, Kubernetes manifests, Kustomize overlays, an Argo CD application, and a GitHub Actions pipeline.

8.2 Testing Strategy

Backend tests use Jest and Supertest. Frontend tests use Jest, Babel, jsdom, and React Testing Library. Local verification performed for this report produced:

- Backend: 9 test suites passed, 45 tests passed.
- Frontend: 5 test suites passed, 14 tests passed.
- Frontend production build: Vite build succeeded.

Table 8.1: Local verification results from July 21, 2026.

Command	Result	Evidence used in report
backend: npm test - -runInBand	9 suites, 45 tests passed	Scoring, workflow, health, visibility, validation, task privacy
frontend: npm test - -runInBand	5 suites, 14 tests passed	Login, app routes, sidebar, objective modal, shared motion
frontend: npm run build	Build succeeded	Vite transformed 881 modules and generated dist

8.3 Backend Test Coverage by Behavior

The backend tests are especially valuable because they protect business rules. The scoring test confirms weighted objective contributions, manager-adjusted achievement, no division of team-objective weight by member count, exclusion of draft and non-final statuses, normalization of incomplete weight totals, and rating-band mapping. Workflow tests check HR review warnings, blocking issues, and bonus/penalty input. Objective-rule tests check allocation buckets and distributed team objectives.

8.4 Frontend Test Coverage by Behavior

Frontend tests cover basic application rendering, login behavior, objective modal interaction, sidebar behavior, and shared motion. This confirms that at least the core shell and selected components are testable. The repository does not include end-to-end tests with a real browser, seeded database, and authenticated workflows.

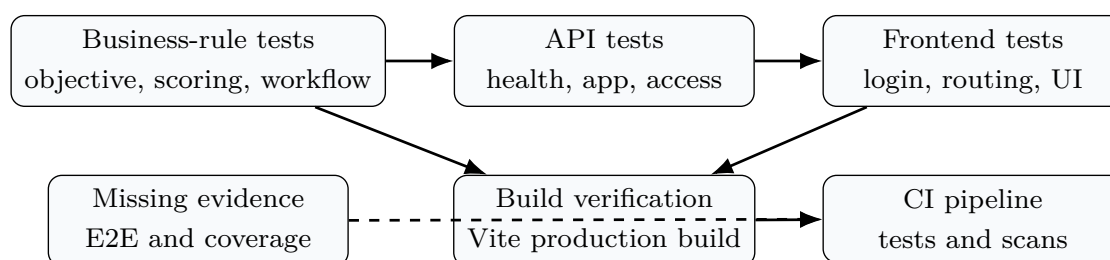


Figure 8.1: Testing strategy map distinguishing verified tests from missing evidence. Source: repository tests and local verification.

[figure 8.1](#) makes the testing boundary explicit. The repository has useful unit, integration, frontend, and build evidence, but no stored coverage percentage and no full browser E2E workflow suite.

8.5 Dockerization

The backend Dockerfile uses `node:22-alpine`, installs production dependencies with `npm ci --omit=dev`, exposes port 5000, and starts `server.js`. The frontend Dockerfile builds the Vite app in a Node stage and serves `dist` through Nginx. The Nginx configuration routes normal SPA paths to `index.html` and proxies `/api` to the backend service.

Docker documentation defines Dockerfiles as text documents containing image build instructions [11]. The implementation follows the common pattern of separating frontend build and static serving.

8.6 Kubernetes and GitOps

Kubernetes manifests define backend and frontend Deployments and ClusterIP Services. The frontend base uses two replicas; the backend base uses one replica and loads environment variables from `backend-secret`. Kustomize overlays exist for dev, QA, staging, and production. Kustomize bases and overlays match the official Kubernetes model for reusable configuration [12].

The Argo CD application points to the GitHub repository path `k8s/base`, target revision `main`, namespace `pfe`, and enables automated sync with prune and self-heal. Argo CD's automated sync model compares desired Git state with live cluster state and can apply changes automatically [13].

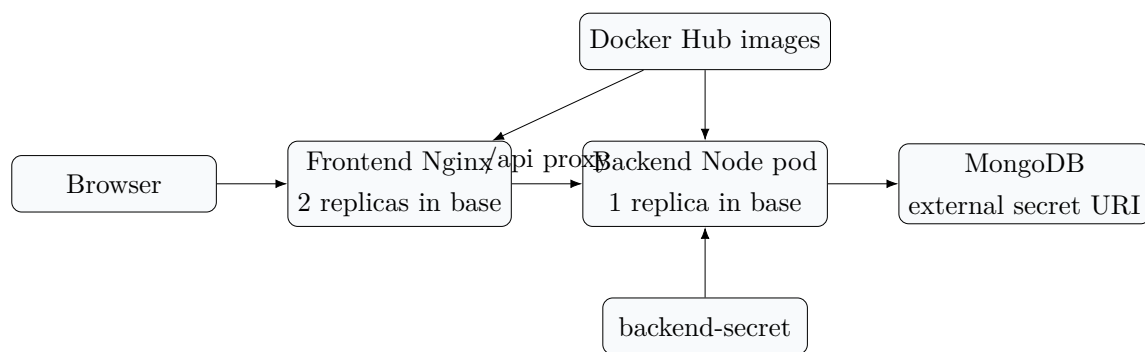


Figure 8.2: Deployment architecture. Source: author's own work based on Docker, Nginx, and Kubernetes manifests.

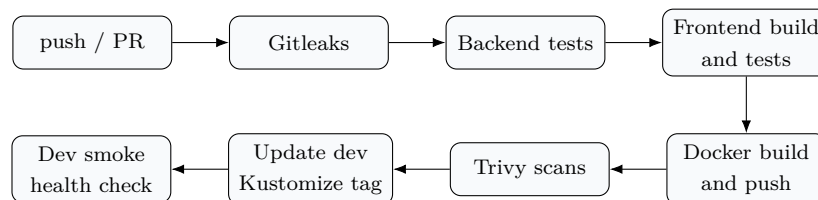


Figure 8.3: CI/CD pipeline diagram. Source: author's own work based on GitHub Actions workflow.

8.7 CI/CD Pipeline

The GitHub Actions workflow runs on push and pull request. It performs:

1. Secret scanning with Gitleaks.
2. Backend dependency installation and tests.
3. Frontend dependency installation, build, and tests.
4. Docker Buildx setup.

5. Docker Hub login on push.
6. Backend and frontend image build.
7. Image push on push events.
8. Development Kustomize image tag update for the `develop` branch.
9. Trivy vulnerability scans for backend and frontend images.
10. Optional smoke health check against `DEV_APP_URL`.

GitHub Actions workflow syntax is the official mechanism used for these repository automations [14].

Table 8.2: CI/CD stages verified from the GitHub Actions workflow.

Stage	Verified action	Quality purpose
Secret scan	Gitleaks action after checkout	Prevent committed secrets from moving deeper into the pipeline
Backend tests	Node 18, <code>npm ci</code> , <code>npm test</code>	Protect API and business-rule behavior
Frontend build/tests	Node 18, Vite build, Jest tests	Confirm SPA compilation and selected UI behavior
Image build/push	Docker Buildx, Docker Hub push on push events	Produce deployable backend and frontend images
GitOps update	Update dev Kustomize tag on <code>develop</code>	Connect image version to deployment configuration
Security scan	Trivy backend and frontend image scans	Block high/critical vulnerabilities when detected
Smoke test	Optional <code>DEV_APP_URL</code> health check	Verify reachable deployed API when configured

8.8 Deployment Limitations

The deployment configuration covers frontend and backend containers, but the Python AI prediction service is not represented in the main Docker Compose or Kubernetes base manifests. The backend depends on secrets such as Mongo URI and JWT secrets, but the report does not expose them. There is no monitoring stack such as Prometheus or Grafana in the repository. No backup or disaster recovery manifest is present beyond demo backup data files.

8.9 Chapter Conclusion

Testing and DevOps evidence is strong for a graduation project. The tests validate important business rules, and the pipeline connects security scanning, build verification, image creation, vulnerability scanning, and GitOps deployment. The main operational gap is incomplete deployment coverage for the AI microservice and missing observability assets.

General Conclusion and Perspectives

PerTrack addresses a concrete HR engineering problem: performance management needs structured objectives, measurable progress, role-based governance, auditable evaluation, and HR validation. The implemented application goes beyond simple record keeping. It models annual cycles, team hierarchy, objective states, KPI tracking, tasks, check-ins, manager evaluation, final HR review, follow-up decisions, analytics, audit logs, and AI support.

The main technical contribution is the combination of workflow logic and score calculation. Objective weights are treated as business rules, not as display metadata. Manager-adjusted achievement can override employee self-reported progress, but the objective breakdown preserves the source values. HR review contains process checks before validation. This design is suitable for sensitive evaluation workflows because it makes decisions traceable.

The project also demonstrates modern implementation practices: React and Vite for the frontend, Express and Mongoose for the backend, Flask and XGBoost for the AI prediction service, JWT authentication, security middleware, structured tests, Docker images, Kubernetes manifests, Kustomize overlays, Argo CD GitOps configuration, and GitHub Actions CI/CD.

The project remains honest about its limitations. Academic and organization metadata is missing from the repository. The AI dataset is synthetic and cannot prove real-world predictive validity. The AI microservice is not fully represented in the container orchestration manifests. Security controls exist, but no penetration test or accessibility audit is present. Some evaluation functionality exists in overlapping modules and should be simplified during future maintenance.

Realistic future work includes single sign-on, stronger object-level authorization audits, accessibility testing, end-to-end tests with seeded data, complete AI service deployment, real organizational training data under strict privacy governance, bias monitoring, explainable prediction outputs, stronger observability, backup policies, richer HRIS integration, and optional mobile support.

Table 8.3: Delivered scope and remaining limitations.

Area	Delivered evidence	Remaining limitation
Core application	Roles, teams, cycles, objectives, tasks, check-ins, evaluations, HR review	Screenshots deferred; no seeded E2E run included
Scoring	Weighted objective score, normalization, rating labels, tests	Requires continued validation with production review cases
AI	LLM draft service, Flask predictor, saved XGBoost artifacts, synthetic dataset metrics	Synthetic data only; no production bias/drift monitoring
DevOps	Docker, Kubernetes, Kustomize, Argo CD, GitHub Actions, Trivy	AI service absent from Compose/K8s base; no monitoring stack
Quality	Backend tests, frontend tests, frontend build, evidence appendices	No coverage report, penetration test, or accessibility audit

Chapter A

API Summary

Table A.1: Grouped API inventory.

Base route	Main methods	Purpose
/api/auth	POST, GET	Login, token refresh, logout, current-user lookup
/api/users	GET, PUT, DELETE	User lists, managers, collaborators, admin user access, profile/avatar update
/api/teams	GET, POST, PUT, DELETE	Team and subteam CRUD, team summaries, managed team access
/api/team-members	GET	Team member lookup and fallback active users
/api/cycles	GET, POST, PUT, PATCH, DELETE	Cycle CRUD, phase advancement, phase check, rollback
/api/objectives	GET, POST, PUT, PATCH, DELETE	Objective CRUD, submit, validate, acknowledge, complete, evaluate, lock, KPIs, comments, progress, change requests
/api/checkins	GET, POST, PUT	Check-in submission, attachments, team review, objective task lookup
/api/tasks	GET, POST, PUT, DELETE	Task creation, assignment, stats, team tasks, time entries
/api/evaluations	GET, POST, PUT	General evaluation creation, scoring, submission, HR approval/rejection, acknowledgment
/api/final-evaluations	GET, POST, PUT	Final evaluation generation, team views, HR queues, HR validation, employee feedback, export

Base route	Main methods	Purpose
/api/hr-decisions	GET, POST, PUT, DELETE	HR decision CRUD and scoped visibility
/api/improvement- plans	GET, POST, PUT, DELETE	Improvement plans linked to final evaluations
/api/bonus-penalty	GET, POST, PUT	Bonus and penalty creation, approval, eligible evaluations
/api/career	GET, POST, PUT, DELETE	Competencies, career paths, development actions, recommendations
/api/ai	GET, POST	Goal/KPI suggestions, review drafts, development plans, prediction endpoints
/api/stats	GET	Dashboard statistics, performance stats, objective status, users by role, score distribution
/api/reports	GET	Cycle and team reports
/api/audit-logs	GET	Audit log queries and entity history
/api/notifications	GET, POST, DELETE	Notifications, unread count, read flags
/api/calendar	GET, POST, DELETE	Calendar providers, connection, callback, events
/api/meetings	GET, POST, PUT, DELETE	Meetings, duplication, action items
/api/pdf	GET	Team and user PDF generation

Chapter B

Database Dictionary

Table B.1: Main MongoDB/Mongoose entities.

Entity	Key fields	Notes
User	name, email, password, role, team, manager, isActive, refreshToken	Email validator requires @biat.com; password hashed before save
Team	name, leader, members, parentTeam, createdBy	Parent-team reference supports subteams
Cycle	name, year, status, phase dates, currentPhase	Year is unique; phase dates sequential
Objective	title, owner, cycle, category, team, assignedUsers, weight, status, KPIs	Embedded KPIs, comments, activity logs, change requests
Task	assignee, assignedBy, status, workflowStage, progress, linkedGoal, timeTracking	Kanban and time tracking support
CheckIn	objective, employee, cycle, progress, status, attachments, history	Manager review and attachment workflow
Evaluation	employeeId, evaluatorId, cycleId, status, suggestedScore, finalScore	General evaluation workflow with approval history

Entity	Key fields	Notes
FinalEvaluation	employee, cycle, auto score, manager score, final score, rating, HR status	Main annual final review and HR validation model
HRDecision	user, cycle, finalEvaluation, finalScore, action	One decision per user and cycle
ImprovementPlan	evaluation, employee, cycle, objective goal, deadline, expected outcome	Follow-up plans after evaluation
BonusPenalty	employee, type, value, reason, finalEvaluation, approvalStatus	Bonus or penalty records
Feedback	sender/recipient fields, type, message, rating	Feedback exchange and stats
Meeting	organizer, attendees, date/time, action items	Meeting management and follow-up
AuditLog	user, action, entityType, entityId, description, metadata, timestamp	Traceability and compliance support
Notification	recipient, type, title, message, read state	In-app notification flow
CareerPath/Recommendation/Opportunity	user, action, entityType, actions, suggested path, skills	Career development module

Chapter C

Frontend Module Inventory

This appendix maps the main frontend modules to the user workflows described in the report. The inventory is based on `frontend/src/routes/routeConfig.jsx`, page files, shared components, API clients, and utility modules.

C.1 Route-Level Pages

Table C.1: Frontend route inventory.

Route or page	Role evidence	Purpose
<code>/login</code>	Public	Authenticates users and creates the initial frontend session state.
<code>/dashboard</code>	Protected	Main role-aware landing page with summaries, charts, alerts, and activity.
<code>/goals</code>	Protected	Objective management, filters, table view, creation and detail modals.
<code>/tasks</code>	Protected	Work-management page with task states, kanban behavior, progress and time tracking.
<code>/calendar</code>	Protected	Calendar page connected to backend provider and event endpoints.
<code>/meetings</code>	Protected	Meeting scheduling, duplication, updates, and follow-up actions.
<code>/feed</code>	Protected	Team/activity feed based on backend feed route.
<code>/cycles</code>	ADMIN	Annual cycle administration and phase control.

Route or page	Role evidence	Purpose
/midyear-assessments	Phase 2	Mid-year workspace guarded by active cycle phase.
/final-evaluations	Phase 3	End-year review workspace for final evaluations.
/performance	ADMIN, HR, TEAM_LEADER	Performance summaries and employee/team score views.
/performance/predictor	ADMIN, HR, TEAM_LEADER	Prediction UI for performance rating and promotion readiness.
/my-team	Protected	Team and subteam visibility for the current user.
/feedback	Protected	Feedback received, sent, and available recipients.
/career	Protected	Career paths, competencies, actions, and recommendations.
/evaluations	ADMIN, HR, COLLABORATOR	Evaluation readiness and general evaluation access.
/validation	TEAM_LEADER	Objective review queue for manager approval.
/hr-validation	ADMIN, HR	HR queue for final evaluation validation and returns.
/hr-decisions	ADMIN, HR	HR follow-up decisions and development actions.
/teams	ADMIN	Team administration.
/users	ADMIN	User administration.
/analytics	ADMIN, HR	Organizational analytics.
/audit-logs	ADMIN, HR	Audit trail consultation.
/bonus-penalty	ADMIN, HR, TEAM_LEADER	Bonus or penalty workflow.
/settings	ADMIN, HR	Settings page.

C.2 Shared Application Shell

The frontend shell is implemented with `DashboardLayout.jsx`, `EnterpriseSidebar.jsx`, and `TopHeader.jsx`. This creates a stable protected layout for role-aware navigation, page title display, active cycle state, search, notifications, and profile menu. The route file groups sidebar items by sections: Main, Annual Cycle, People, Development, and Management.

C.3 Authentication and API Clients

`AuthContext.jsx` owns session behavior. It reads and writes access tokens, refresh tokens, and a compact current-user object from browser storage. It pings `/auth/me`, refreshes access tokens through `/auth/refresh`, and clears the session on logout or invalid refresh. `frontend/src/services/api.js` creates the main Axios instance, adds authorization headers, retries eligible 401 responses once after refreshing the token, caches selected GET requests, and clears cached GET responses after mutations.

Table C.2: Frontend service and utility modules.

Module	Responsibility
<code>services/api.js</code>	Main Axios client, bearer token injection, refresh retry, GET request caching and prefetching.
<code>api/apiClient.js</code>	Secondary Axios wrapper used by small domain API helper files.
<code>api/cycles.js</code>	Convenience functions for cycle API calls.
<code>api/teams.js</code>	Convenience functions for team API calls.
<code>api/users.js</code>	User filtering helpers.
<code>api/hrDecisions.js</code>	HR decision API helper functions.
<code>api/ai.js</code>	AI and prediction-related API calls.
<code>hooks/useActiveCycle.js</code>	Active cycle lookup for phase-aware screens.
<code>hooks/usePersistentTimer.js</code>	Persistent task timer behavior.
<code>utils/objectiveRules.js</code>	Frontend helper for objective availability/visibility state.
<code>utils/workManagement.js</code>	Normalizes task, calendar, check-in, objective, and productivity structures.

C.4 Goal and Objective UI

The objective UI is distributed across `GoalsPage.jsx` and components under `components/goals`. The creation modal includes weight selection and capacity awareness. The detail panel supports tabs for details, KPIs, updates, comments, and activity. The table exposes status, category, weight, progress, KPI count, and expanded details. Manager review and evaluate modals separate approval/review actions from ordinary editing.

C.5 Work Management UI

The task page and task components implement a compact operational workspace. The model and UI support task status, workflow stage, progress percentage, priority, due date,

linked objective, team, recurrence, and time tracking. This module matters for the final evaluation because task completion and check-in evidence are used by scoring/evidence services.

C.6 Evaluation UI

Final evaluation views are separated between employee and manager perspectives. The manager view supports generation, update, recalculation, score adjustment, HR submission, and AI-assisted draft use. HR validation has its own page so HR can review pending and returned items without mixing manager-owned editing responsibilities.

C.7 UI States

The repository contains shared `EmptyState`, `LoadingSkeleton`, `ErrorBoundary`, `Toast`, `Badge`, and `ConfirmDialog` components. This indicates that loading, empty, error, confirmation, and feedback states were considered as reusable product behaviors rather than repeated inline messages.

Chapter D

Business Rule Catalog

This appendix records the main business rules verified in the code. It is intentionally explicit because performance-management applications are judged less by screen count than by rule correctness.

D.1 Role and Identity Rules

Table D.1: Role and identity rules.

Rule	Implementation	Evidence
Only four roles are accepted	ADMIN, HR, TEAM_LEADER, COLLABORATOR	backend/models/User.js
User emails must match the organization domain	Email validator requires @biat.com	backend/models/User.js
Passwords are not stored in clear text	Pre-save hook hashes modified password fields with bcrypt salt rounds	backend/models/User.js
Admin passes route role checks	role.js returns next() for ADMIN	backend/middleware/role.js
Inactive or deleted users lose API access	Auth middleware loads current user and checks isActive and isDeleted	backend/middleware/auth.js

D.2 Team and Scope Rules

Table D.2: Team and access-scope rules.

Rule	Implementation	Evidence
Teams can contain subteams	<code>parentTeam</code> points from one team to another	<code>backend/models/Team.js</code>
Managed team scope is recursive	Helper gathers root teams and child teams into a managed set	<code>backend/utils/accessControl.js</code>
Team leaders can access managed employees	Managed employee ids combine team members and direct reports	<code>backend/utils/accessControl.js</code>
Root team leaders are constrained	Route validation prevents conflicting manager assignment	<code>backend/routes/teams.js</code>
Team objective copies are deduplicated by team, cycle, and title	Group key avoids counting distributed copies multiple times	<code>backend/utils/objectiveRules.js</code>

D.3 Cycle Rules

Table D.3: Cycle and phase rules.

Rule	Implementation	Evidence
Cycle year is unique	Unique index on <code>year</code>	<code>backend/models/Cycle.js</code>
Phase dates must be sequential	Pre-save validation compares provided phase dates	<code>backend/models/Cycle.js</code>
Closed cycles cannot be edited normally	Update route rejects edits to closed cycles	<code>backend/routes/cycles.js</code>
Phase advancement is sequential	Route prevents jumping forward or moving backward except rollback rule	<code>backend/routes/cycles.js</code>
Rollback is limited	Rollback from <code>phase2</code> to <code>phase1</code> is admin-only and blocked after submitted assessments	<code>backend/routes/cycles.js</code>
Closing a cycle changes status to closed	Phase patch route maps <code>closed</code> phase to <code>closed</code> status	<code>backend/routes/cycles.js</code>

D.4 Objective Workflow Rules

Table D.4: Objective workflow and visibility rules.

Rule	Implementation	Evidence
Objective weight must be between 1 and 100	Joi and Mongoose validation enforce numeric bounds	<code>backend/validators/schemas.js</code> , <code>Objective.js</code>
Cancelled and archived objectives do not consume allocation	Weight helper excludes these statuses	<code>backend/utils/objectiveRules.js</code>
Rejected objectives still count in planning weight	Tests verify rejected is weight-bearing	<code>backend/tests/objectiveRules.test.js</code>
Private draft individual objectives are owner-only	Visibility helper hides employee-created draft individual goals from non-owners	<code>backend/utils/objectiveVisibility.js</code>
Manager-assigned objectives require acknowledgment	Objective schema includes <code>assigned</code> and <code>acknowledged</code> states	<code>backend/models/Objective.js</code>
Final-score objective statuses are limited	Only approved, validated, evaluated, and locked objectives are scored	<code>backend/services/scoreCalculationService.js</code>

D.5 Scoring Rules

Table D.5: Scoring rules.

Rule	Implementation	Evidence
Objective weighted points equal weight times achievement divided by 100	Service and objective pre-save hook use the same formula	<code>Objective.js</code> , <code>scoreCalculationService.js</code>

Rule	Implementation	Evidence
Manager confirmation has priority	Achievement uses manager-adjusted percent before final self percent and ordinary progress	<code>scoreCalculationService.js</code>
Team objective weight is not divided by member count	Tests assert full team weight is applied to each assigned member	<code>scoreCalculationService.test.js</code>
Incomplete historical totals are normalized	If total eligible weight is not 100, raw points are divided by total weight	<code>scoreCalculationService.js</code>
Scores are clamped to 0–100	<code>clampPercentage</code> normalizes input values	<code>scoreCalculationService.js</code>
Rating bands are deterministic	Five label thresholds are implemented	<code>scoreCalculationService.js</code>

D.6 HR Review Rules

Table D.6: HR review and follow-up rules.

Rule	Implementation	Evidence
Only submitted final evaluations enter HR validation	Blocking rule checks <code>pending_hr</code> status	<code>backend/utils/workflowRules.js</code>
Manager-owned report fields are mandatory before HR validation	Blocking issues require score, rating, comments, strengths, weaknesses, and breakdown	<code>workflowRules.js</code>
Large manual score differences need justification	Warning checks score difference of at least 10 points without manager justification	<code>workflowRules.js</code>
Low scores should have an improvement plan	Warning checks final score below 50 with no plan context	<code>workflowRules.js</code>
AI-assisted drafts require manager review confirmation	Blocking issue rejects unreviewed AI-assisted drafts	<code>workflowRules.js</code>

Rule	Implementation	Evidence
Bonus/penalty values must be positive and justified	Input validation requires valid type, positive value, and reason	<code>workflowRules.js</code>

Chapter E

Security Review

This appendix summarizes implemented security controls and residual risks. It is not a penetration-test report; it is a code-based review.

E.1 Implemented Controls

Table E.1: Implemented security controls.

Control area	Implementation	Evidence
Password storage	Bcrypt hash with salt rounds in pre-save hook	<code>backend/models/User.js</code>
Access tokens	Bearer JWT verified with <code>JWT_SECRET</code>	<code>backend/middleware/auth.js</code>
Refresh tokens	Login and refresh endpoints issue and rotate refresh tokens	<code>backend/routes/auth.js</code>
Role enforcement	Route middleware rejects users outside allowed roles	<code>backend/middleware/role.js</code>
Object access	Helper functions check employee, team, and objective access	<code>backend/utils/accessControl.js</code>
HTTP headers	Helmet enabled with adjusted cross-origin resource policy	<code>backend/app.js</code>
CORS	Allowed origins list for local app URLs and credentials	<code>backend/app.js</code>
Rate limiting	Global rate limit and route rate-limiter integration	<code>backend/app.js</code> , <code>middleware/rateLimiter.js</code>
XSS sanitization	<code>xss-clean</code> middleware	<code>backend/app.js</code>

Control area	Implementation	Evidence
NoSQL injection mitigation	<code>express-mongo-sanitize</code> middleware	<code>backend/app.js</code>
Input validation	Joi schemas on selected routes	<code>backend/validators/schemas.js</code>
Upload handling	Multer and static upload restrictions; check-in uploads blocked from direct static access	<code>backend/app.js</code> , <code>backend/routes/checkins.js</code>
Secrets in CI	Gitleaks secret scan before tests and builds	<code>.github/workflows/docker-build.yml</code>

E.2 Authentication Risks

Tokens are stored in browser `localStorage`. This is simple and common in student projects, but it increases exposure if a cross-site scripting bug reaches the page. The backend applies XSS cleaning and Helmet, yet the safer long-term design would be secure, HTTP-only cookies or a carefully reviewed token storage strategy. The current implementation also depends on secret strength and correct production environment management.

E.3 Authorization Risks

Role-based access is visible in routes, but role checks alone do not solve object-level authorization. The project partly addresses this with access helpers and tests for objective visibility and task privacy. The remaining risk is uneven enforcement across a large route surface. A future security pass should review every endpoint with the question: can a user change the identifier in the URL or body to access another employee's data?

E.4 Data Privacy Risks

The platform handles sensitive HR data: evaluations, comments, scores, improvement plans, bonus/penalty records, and employee feedback. The report does not include personal data, secrets, or real employee content. For production, the system would need retention policies, access logging review, encryption-at-rest verification, backup protection, and formal HR data-processing rules.

E.5 AI Privacy Risks

Generative AI requests send compacted performance context to an external provider depending on `AI_PROVIDER`. The code limits input size and asks the provider to use supplied data only, but it does not by itself guarantee provider-side privacy. A production deployment should document which provider is used, what data is sent, whether logs are retained, and how employees are informed.

E.6 Deployment Risks

Kubernetes manifests reference `backend-secret`, but the repository does not include secret templates. That is appropriate because secrets should not be committed. However, deployment repeatability would improve with a sanitized environment-variable template and secret-management instructions. The CI workflow includes Trivy scans, but monitoring, alerting, and backup automation are not present.

E.7 Residual Risk Summary

Table E.2: Residual security risks and recommendations.

Risk	Severity	Recommendation
Token theft through frontend XSS	High	Review token storage and consider HTTP-only secure cookies.
Uneven object-level authorization	High	Add endpoint-level authorization tests for every employee-scoped route.
Unreviewed AI data transfer	High	Add AI data-processing policy, opt-in controls, and provider-specific privacy review.
Missing observability	Medium	Add structured logs, metrics, health dashboards, and alerting.
No formal backup plan	Medium	Define MongoDB backup, restore, and retention policy.
No accessibility audit	Medium	Add keyboard, screen-reader, contrast, and responsive QA checks.

Chapter F

AI Model Evidence

This appendix records the AI prediction evidence computed from the included dataset and saved artifacts. The values describe the repository’s synthetic dataset and should not be interpreted as validation on real HR data.

F.1 Dataset Fields

Table F.1: AI dataset and prediction fields.

Field	Used by model	Meaning
kpi_score	Yes	Numeric KPI achievement score between 0 and 100.
goal_completion_percent	Yes	Percentage of goals completed.
checkin_count	Yes	Number of submitted check-ins, bounded by the API between 0 and 20.
avg_checkin_progress	Yes	Average progress reported in check-ins.
feedback_count	Yes	Number of feedback items.
positive_feedback_ratio	Yes	Ratio between 0 and 1.
task_completion_percent	Yes	Percentage of assigned tasks completed.
tasks_on_time_percent	Yes	Percentage of tasks completed on time.
overall_score	No	Deterministic score excluded from training to avoid leakage.
rating	Target	Four-class performance rating target.
promotion_ready	Target	Binary promotion-readiness target.
strengths, weaknesses, review_summary, suggestions	No	Generated text fields used for realistic API outputs.

F.2 Dataset Quality Checks

Table F.2: Dataset checks computed locally.

Check	Value
Rows	10,000
Missing values	0
Duplicate rows	0
Rating classes	4
Prediction input features	8

F.3 Class Distribution

Table F.3: Rating class distribution in the synthetic dataset.

Rating	Count
<code>exceptional</code>	2,238
<code>exceeds_expectations</code>	2,722
<code>meets_expectations</code>	2,440
<code>needs_improvement</code>	2,600

F.4 Promotion Readiness by Rating

Table F.4: Promotion-readiness rate by rating in the synthetic data.

Rating	Promotion-ready rate
<code>exceptional</code>	0.7962
<code>exceeds_expectations</code>	0.5103
<code>meets_expectations</code>	0.2102
<code>needs_improvement</code>	0.0246

F.5 Rating Confusion Matrix

Table F.5: XGBoost rating model confusion matrix on the repository split.

Actual / predicted	exceeds	exceptional	meets	needs improvement
exceeds expectations	525	12	7	0
exceptional	5	443	0	0
meets expectations	12	0	475	1
needs improvement	0	0	7	513

F.6 Promotion Confusion Matrix

Table F.6: XGBoost promotion model confusion matrix on the repository split.

Actual / predicted	not ready	ready
not ready	1005	245
ready	259	491

F.7 Feature Importance

Table F.7: XGBoost rating feature importance from saved artifact.

Feature	Importance
kpi_score	0.6108
task_completion_percent	0.1700
goal_completion_percent	0.1085
avg_checkin_progress	0.0535
positive_feedback_ratio	0.0360
tasks_on_time_percent	0.0105
checkin_count	0.0061
feedback_count	0.0046

F.8 Interpretation

The rating classifier performs very well because the synthetic target is strongly linked to the same input features that define the deterministic overall score. The promotion classifier performs more modestly because the dataset intentionally includes probabilistic promotion readiness. This is realistic as a demonstration, but it also means the service

should be presented as an educational decision-support feature until real organizational validation exists.

Chapter G

Installation Guide

G.1 Prerequisites

The project requires Node.js, npm, MongoDB access, Python with the AI dependencies when the prediction service is used, Docker for container builds, and Kubernetes tooling for cluster deployment.

G.2 Backend

```
cd backend
npm ci
# create .env with MONGO_URI, JWT_SECRET, JWT_REFRESH_SECRET
npm run dev
```

G.3 Frontend

```
cd frontend
npm ci
npm run dev
```

G.4 AI Service

```
cd ai-service
pip install -r requirements.txt
python app.py
```

G.5 Docker Compose

```
docker compose up --build
```

The Compose file builds the backend and frontend only. The AI service must be run separately unless deployment files are extended.

Chapter H

Test Evidence

H.1 Backend Tests

Local command:

```
cd backend
npm test -- --runInBand
```

Result: 9 test suites passed, 45 tests passed.

H.2 Frontend Tests

Local command:

```
cd frontend
npm test -- --runInBand
```

Result: 5 test suites passed, 14 tests passed.

H.3 Frontend Build

Local command:

```
cd frontend
npm run build
```

Result: Vite build succeeded and transformed 881 modules.

H.4 AI Artifact Evaluation

The saved AI models were evaluated against the repository dataset split with `random_state=42` and stratification. The XGBoost rating model achieved 0.9780

accuracy. The XGBoost promotion model achieved 0.7480 accuracy and 0.8218 ROC-AUC. These metrics describe the included synthetic dataset only.

Bibliography

- [1] “Databases and collections in mongodb,” MongoDB, Accessed: Jul. 21, 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/core/databases-and-collections/>
- [2] “Mongoose schemas guide,” Mongoose, Accessed: Jul. 21, 2026. [Online]. Available: <https://mongoosejs.com/docs/guide.html>
- [3] “React documentation,” Meta Open Source, Accessed: Jul. 21, 2026. [Online]. Available: <https://react.dev/>
- [4] “Vite guide,” Vite, Accessed: Jul. 21, 2026. [Online]. Available: <https://vite.dev/guide/>
- [5] “Express.js routing guide,” OpenJS Foundation, Accessed: Jul. 21, 2026. [Online]. Available: <https://expressjs.com/en/5x/guide/routing/>
- [6] “Flask quickstart,” Pallets, Accessed: Jul. 21, 2026. [Online]. Available: <https://flask.palletsprojects.com/en/stable/quickstart/>
- [7] “Xgboost documentation,” XGBoost, Accessed: Jul. 21, 2026. [Online]. Available: <https://xgboost.readthedocs.io/>
- [8] “Randomforestclassifier,” scikit-learn, Accessed: Jul. 21, 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [9] M. Jones, J. Bradley, and N. Sakimura, *Rfc 7519: Json web token (jwt)*, IETF, 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [10] “Owasp api security top 10 2023,” OWASP, Accessed: Jul. 21, 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [11] “Dockerfile reference,” Docker, Accessed: Jul. 21, 2026. [Online]. Available: <https://docs.docker.com/reference/dockerfile/>
- [12] “Declarative management of kubernetes objects using kustomize,” Kubernetes, Accessed: Jul. 21, 2026. [Online]. Available: <https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>

- [13] “Argo cd automated sync policy,” Argo CD, Accessed: Jul. 21, 2026. [Online]. Available: https://argo-cd.readthedocs.io/en/stable/user-guide/auto_sync/
- [14] “Workflow syntax for github actions,” GitHub, Accessed: Jul. 21, 2026. [Online]. Available: <https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions>